

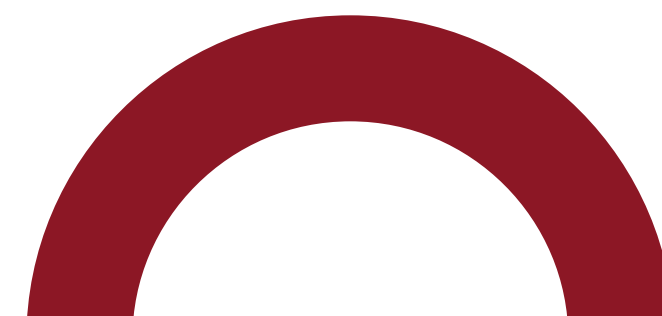


2026-1 BASIC STUDY ML 5주차

Dimension Reduction, SVM

SESSION LEADER: 20기 박준희

2026.02.05



과제 우수자 발표

윤지원 님

화면 공유해주시고
5분 이내로 설명 부탁드립니다 :)



Curse of Dimensionality

Keywords

Dimensionality

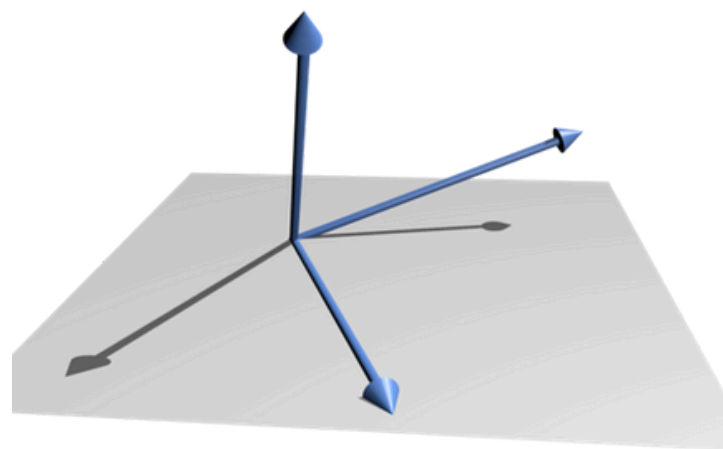
Curse of Dimensionality

Reasons and Solutions

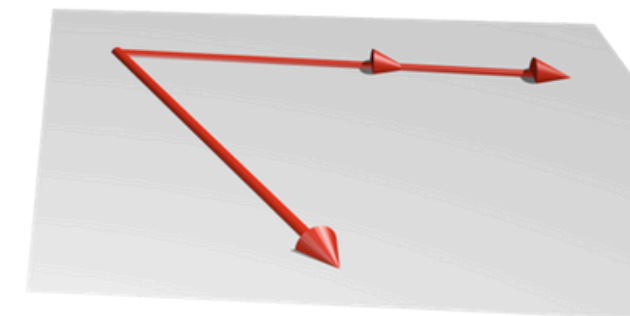
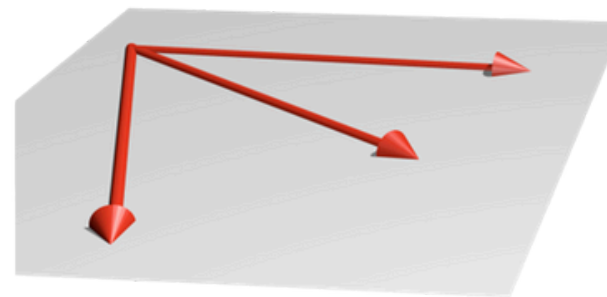
What is Dimensionality?

- 명확하게 정의하기는 어려움
- 분야마다, 조건마다 정의가 다르기 때문
- 선형 대수에서의 dimension 정의를 이용해보자

→ 선형적으로 독립인 (linearly independent) 벡터의 수



Linearly Independent
→ 3차원



Linearly Dependent (같은 평면에 존재)
→ 2차원

Invertible Matrix Theorem

- **Linear Independence (선형독립) 이란?**

- $v_1, v_2, \dots, v_k$: 벡터공간 V 에 존재하는 벡터
- $a_1, a_2, \dots, a_k$: 스칼라
- 0 : 영벡터

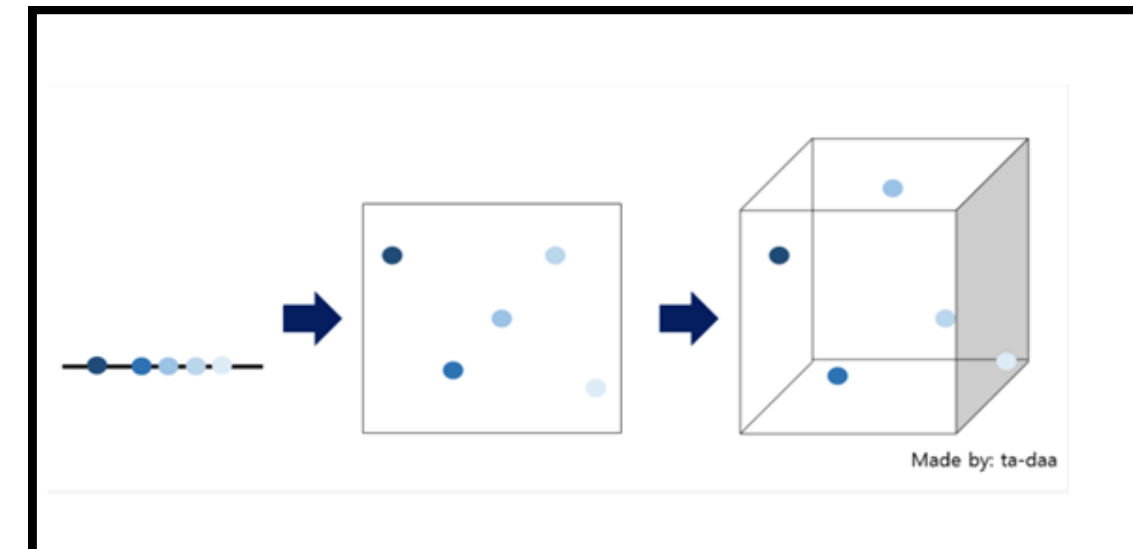
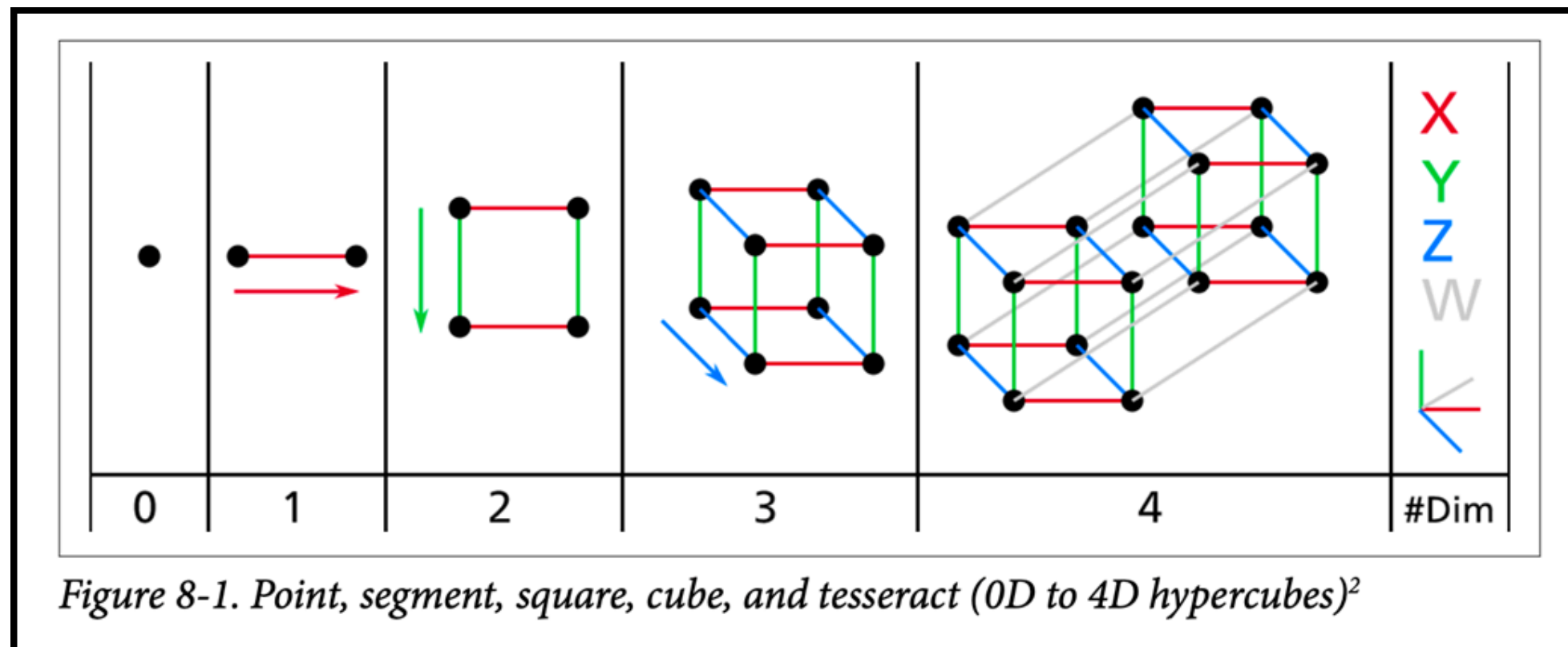
$$a_1 v_1 + a_2 v_2 + \dots + a_k v_k = \mathbf{0}$$

→ $a_1, a_2, \dots, a_k$ 중에서 적어도 0이 아닌 해가 1개 있다면 Linearly Dependent

Curse of Dimensionality

As dimensionality increases..

- data becomes increasingly sparse in the data space
- most training instances are likely to be far away from each other
- New instance will be likely be far away from training instance → overfitting



Curse of Dimensionality

Why does it happen?

Recap) 여러 가지 distance

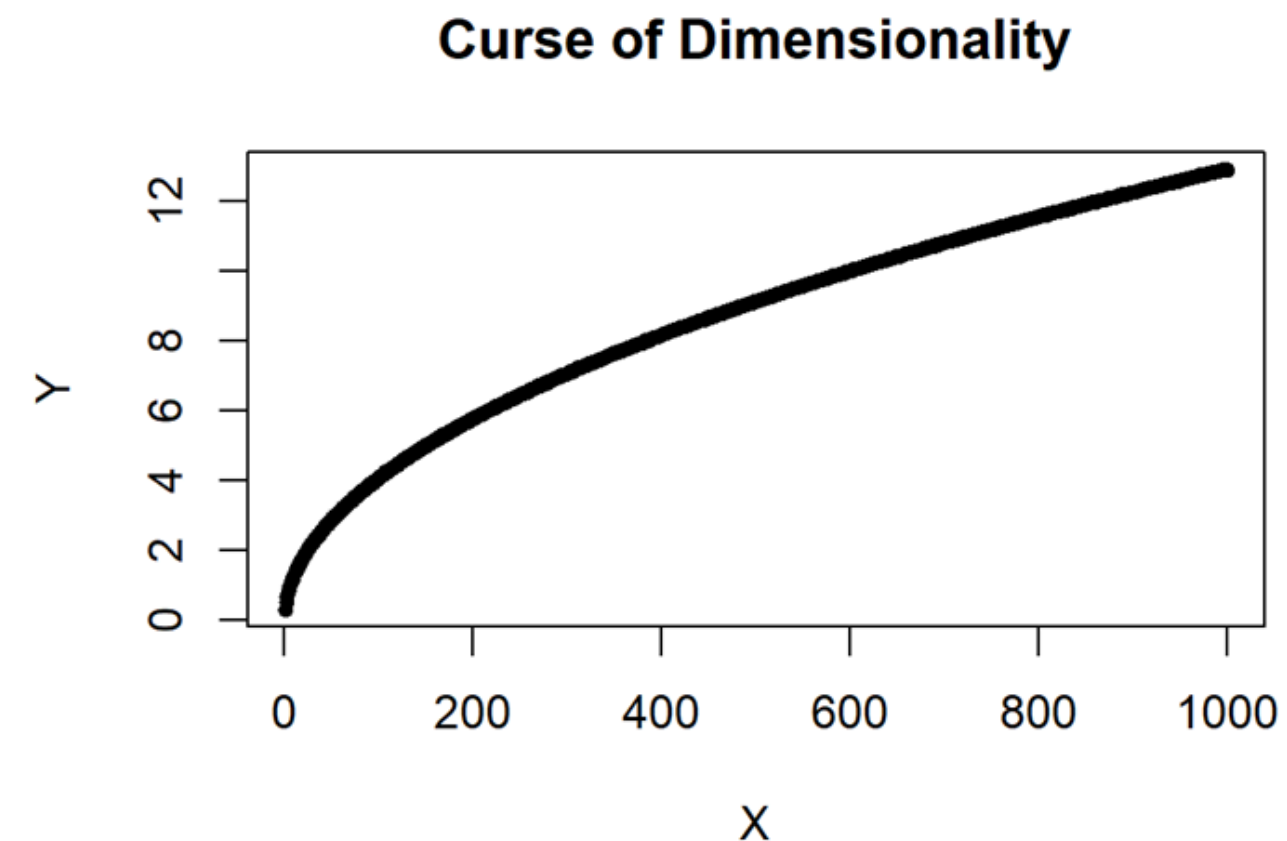
- Distance measure

$$d(\mathbf{u}, \mathbf{v}) = (\sum |u_i - v_i|^2)^{\frac{1}{2}} = \|\mathbf{u} - \mathbf{v}\|_2 \quad \text{Euclidean (L2 norm)}$$

$$d(\mathbf{u}, \mathbf{v}) = \sum |u_i - v_i| = \|\mathbf{u} - \mathbf{v}\|_1 \quad \text{Manhattan (L1 norm)}$$

$$d(\mathbf{u}, \mathbf{v}) = (\sum |u_i - v_i|^p)^{\frac{1}{p}} = \|\mathbf{u} - \mathbf{v}\|_p \quad \text{Minkowski (Lp norm)}$$

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})} \quad \text{Mahalanobis Distance}$$



Dimension 증가 → 항 개수 증가 → Distance 증가

→ 거리는 차원에 대해 단조증가

Curse of Dimensionality

Solution

- Increase Size N
 - the number of training instances required to reach a given density grows exponentially with the number of dimensions
- Feature Selection
 - Choosing $k < d$ important features, ignoring the remaining $d - k$
 - Subset selection algorithms (Ex. LASSO)
- Feature Extraction
 - Project the original x_i , $i = 1, \dots, d$ dimensions to new $k < d$ dimensions, z_j , $j = 1, \dots, k$
 - Ex. PCA
- Regularization
 - Ex. Ridge regression

Why Reduce Dimensionality?

- Reduces time complexity : Less computation
- Reduces space complexity : Fewer parameters
- Saves the cost of observing the feature
- Simpler models are more robust on small datasets : more General model
- More interpretable : simpler explanation
- Enables Data visualization (structure, groups, outliers, etc) if plotted in 2 or 3 dimensions

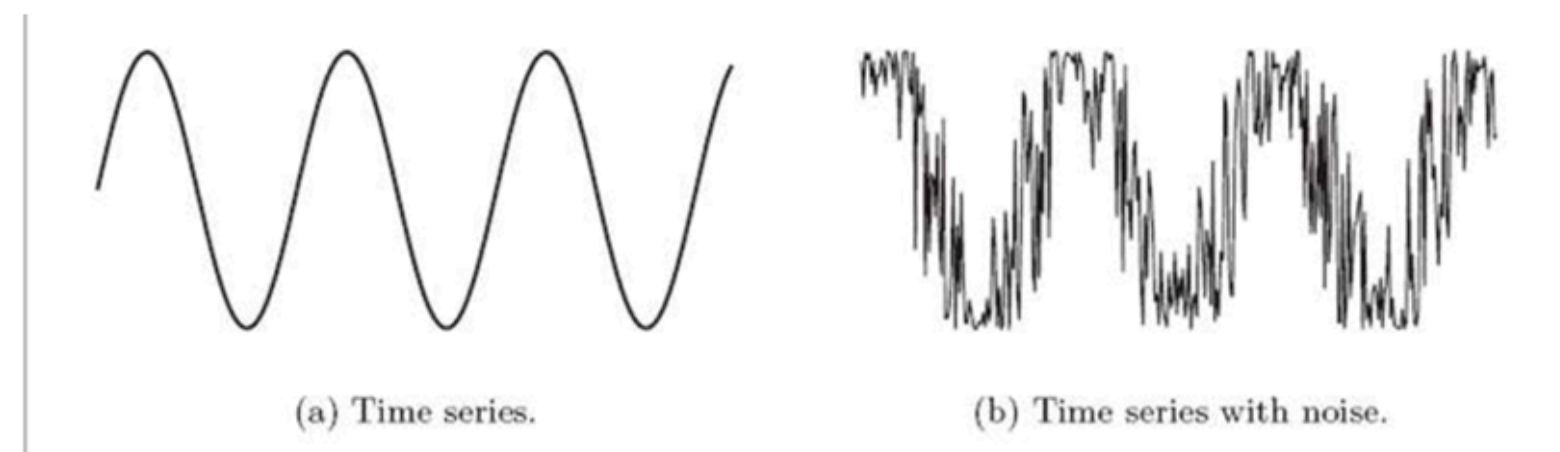
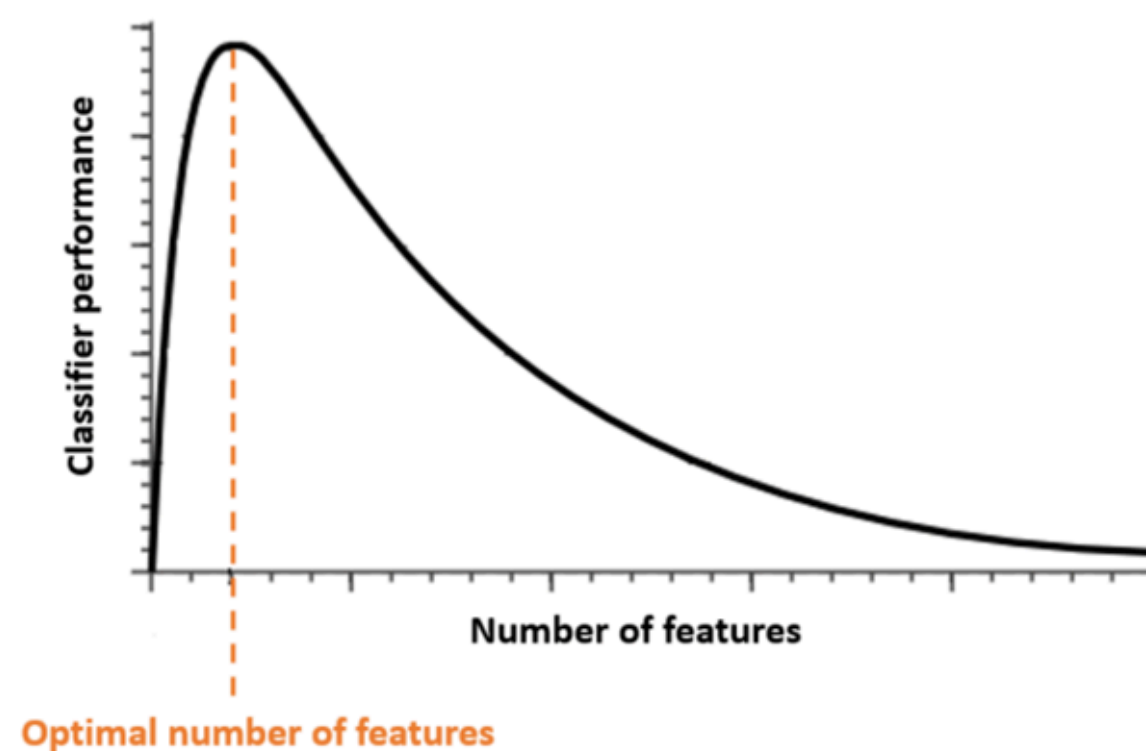


Figure 2.5. Noise in a time series context.

In Conclusion..

차원축소를 하는 이유는

- 1. Curse of Dimensionality를 완화하기 위함
- 2. 우리는 3차원보다 높은 차원을 이해하기 어렵기 때문에
- 3. 단순선형회귀의 경우 $N < p$ (데이터수 < 변수 개수)면 다음의 해는 존재하지 않음
∴ Invertible matrix theorem와 $\text{rank}(X^T X) = \text{rank}(X)$ 에 의해

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\beta}$$



Algorithms

Keywords

PCA

LDA, QDA

Eigenvalue and Eigenvector

Matrix we are finding the
eigenvector/eigenvalue of

eigenvalue

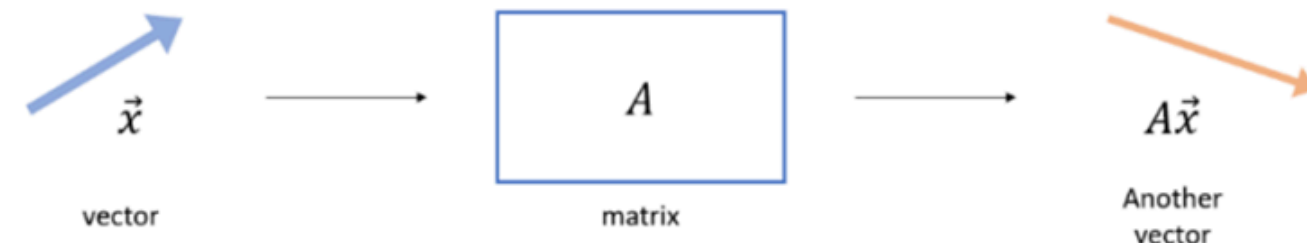
$$Ax = \lambda x$$

$$(A - \lambda I)x = 0$$

identity matrix

For matrix A, scalar lambda which does
not change the direction of x,
is called "eigenvalue" of A

corresponding vector is called
"eigenvector" x



$\det(A - \lambda I) = 0$ 의 해가 eigenvalue $\Rightarrow \lambda$ 구함
 $Ax = \lambda x$ 에서 0이 아닌 해가 eigenvector $\Rightarrow x$ 구함

PCA (“Principal Component Analysis”)

Covariance and Correlation Matrix

변수1 (X1)	변수2 (X2)	변수3 (X3)	변수4 (X4)
x11	x12	x13	x14
x21	x22	x23	x24
x31	x32	x33	x34
x41	x42	x43	x44

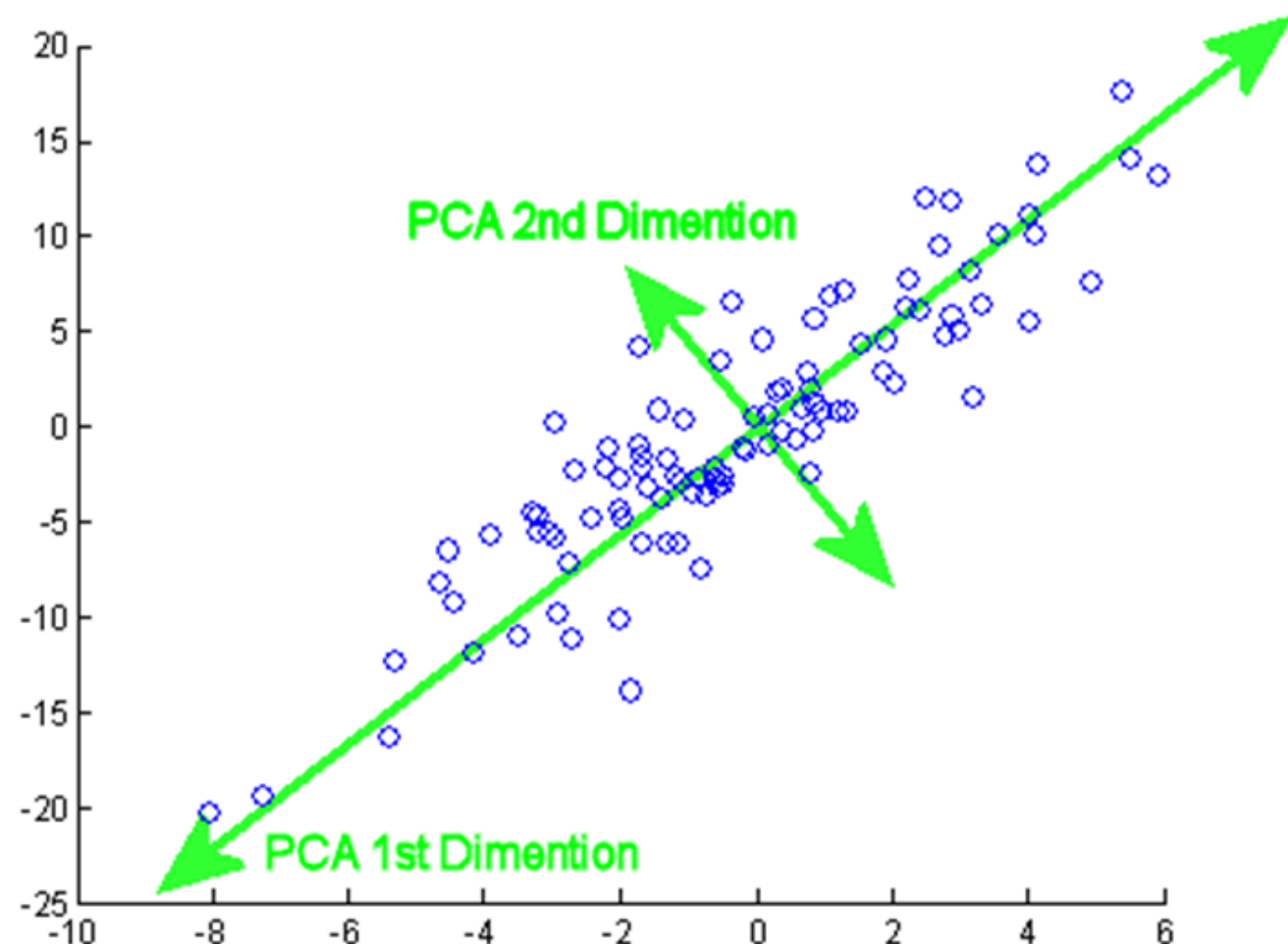
Covariance Matrix

$$\begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \text{Cov}(X_1, X_3) & \text{Cov}(X_1, X_4) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \text{Cov}(X_2, X_3) & \text{Cov}(X_2, X_4) \\ \text{Cov}(X_3, X_1) & \text{Cov}(X_3, X_2) & \text{Cov}(X_3, X_3) & \text{Cov}(X_3, X_4) \\ \text{Cov}(X_4, X_1) & \text{Cov}(X_4, X_2) & \text{Cov}(X_4, X_3) & \text{Cov}(X_4, X_4) \end{bmatrix}$$

Correlation Matrix

$$\begin{bmatrix} \text{Cor}(X_1, X_1) & \text{Cor}(X_1, X_2) & \text{Cor}(X_1, X_3) & \text{Cor}(X_1, X_4) \\ \text{Cor}(X_2, X_1) & \text{Cor}(X_2, X_2) & \text{Cor}(X_2, X_3) & \text{Cor}(X_2, X_4) \\ \text{Cor}(X_3, X_1) & \text{Cor}(X_3, X_2) & \text{Cor}(X_3, X_3) & \text{Cor}(X_3, X_4) \\ \text{Cor}(X_4, X_1) & \text{Cor}(X_4, X_2) & \text{Cor}(X_4, X_3) & \text{Cor}(X_4, X_4) \end{bmatrix}$$

PCA (“Principal Component Analysis”)



데이터 행렬 X 가 $n \times p$ 행렬이라고 하자

- Correlation matrix, Covariance matrix 둘 중 하나를 고른다
 - 주로 correlation matrix를 고름 $\rightarrow \Sigma$ 라고 하자. ($p \times p$)
- Σ 의 eigenvalue와 eigenvector를 구한다. $\rightarrow$ 각각 p 개
- eigenvalue를 $\lambda_1 \geq \lambda_2 \cdots \geq \lambda_p$ 로 정렬하고, 각각에 대응하는 eigenvector를 $v_1, v_2, \cdots v_p$ 라고 하자.
- $X = [x_1 \ x_2 \ \cdots \ x_p]$ 데이터를 변수들의 벡터로 생각할 때:

$$\begin{aligned} \text{PC1} &= v_1^T X, \\ \text{PC2} &= v_2^T X \dots \end{aligned}$$

How Many Dimensions?

W : eigenvector of Σ (covariance matrix of x)

α : eigenvalue of Σ (covariance matrix of x)

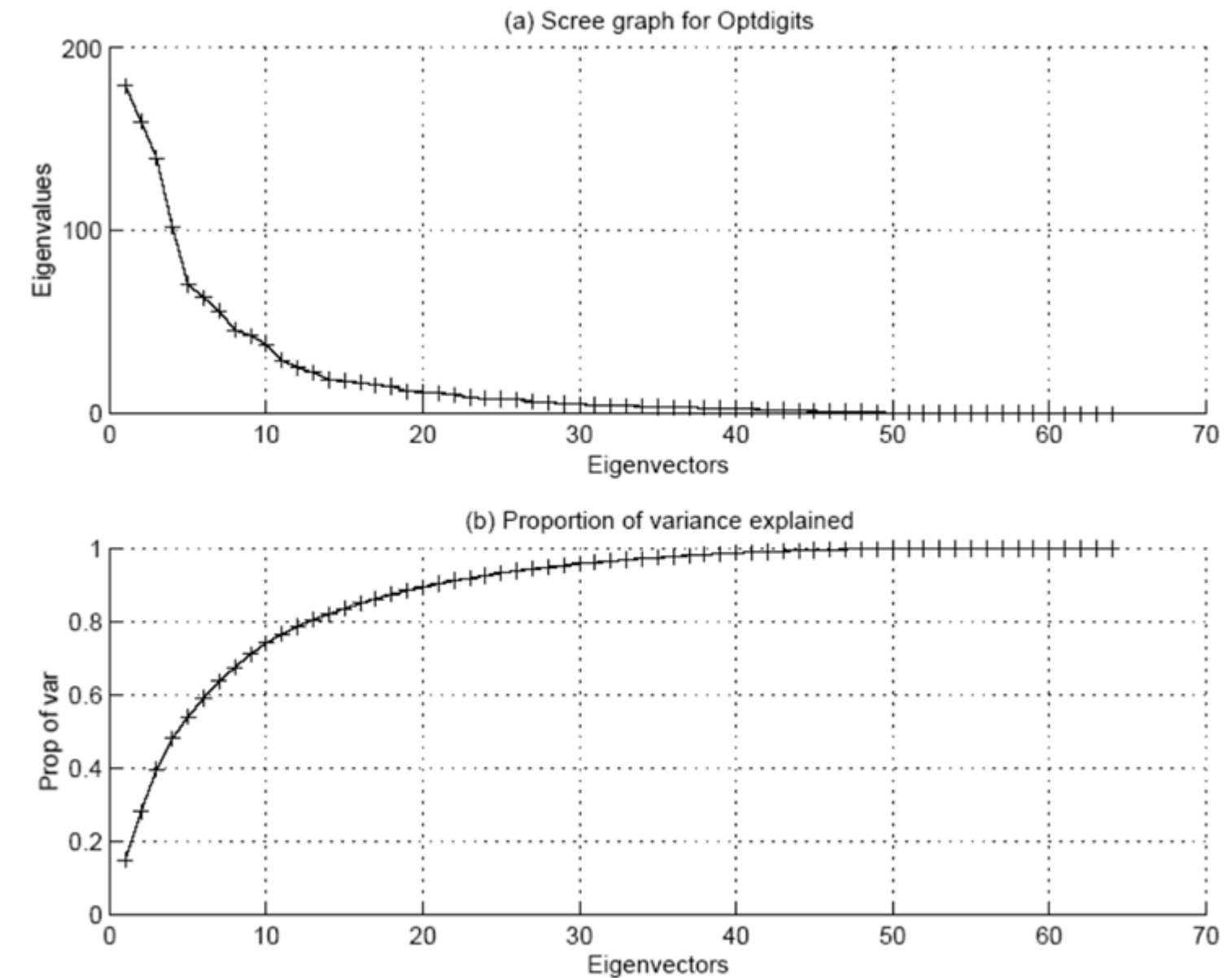
$\rightarrow \Sigma$: $d \times d$ matrix $\rightarrow$ we can find W & α for all d vectors

- Proportion of Variance (PoV)

$$\frac{\alpha_1 + \alpha_2 + \dots + \alpha_k}{\alpha_1 + \alpha_2 + \dots + \alpha_k + \dots + \alpha_d}$$

$$\alpha_1 \geq \alpha_2 \geq \dots \geq \alpha_k \dots \geq \alpha_d$$

- Typically, stop at PoV > 0.9
- Scree graph plots PoV vs k , stop at elbow



Example : PCA

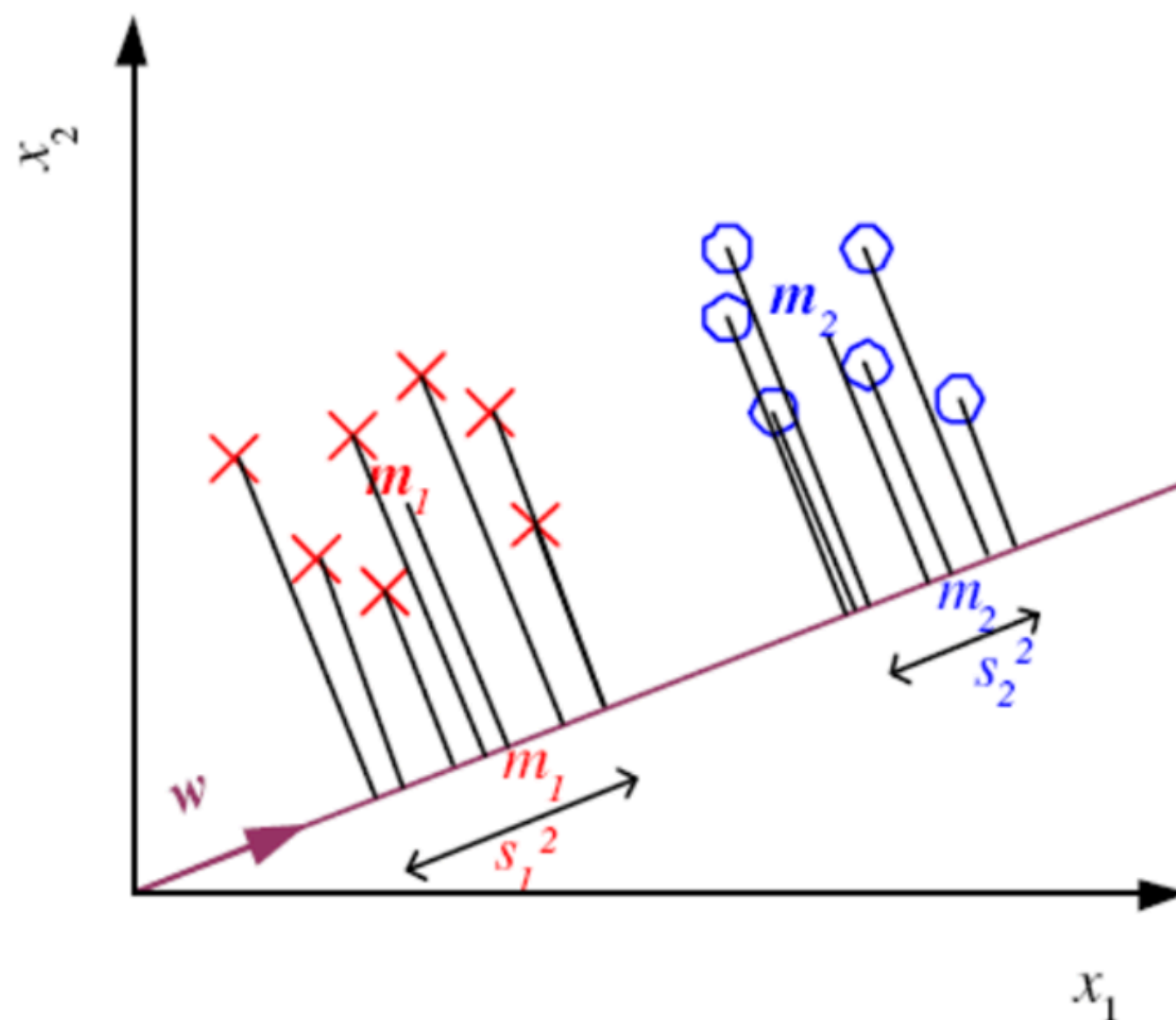
Dimensions = 206 	Dimensions = 160 	Dimensions = 120 
Dimensions = 80 	Dimensions = 40 	Dimensions = 10 

LDA (“Linear Discriminant Analysis”)

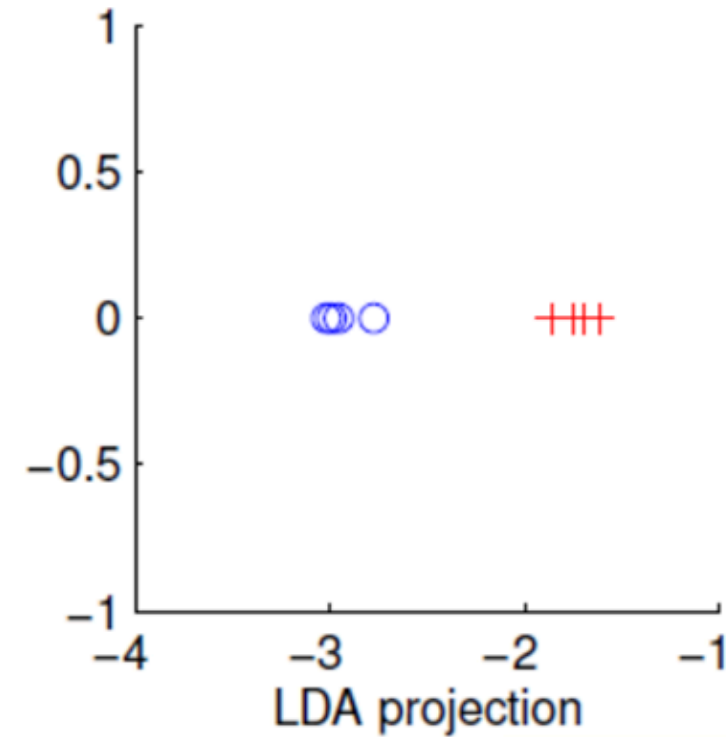
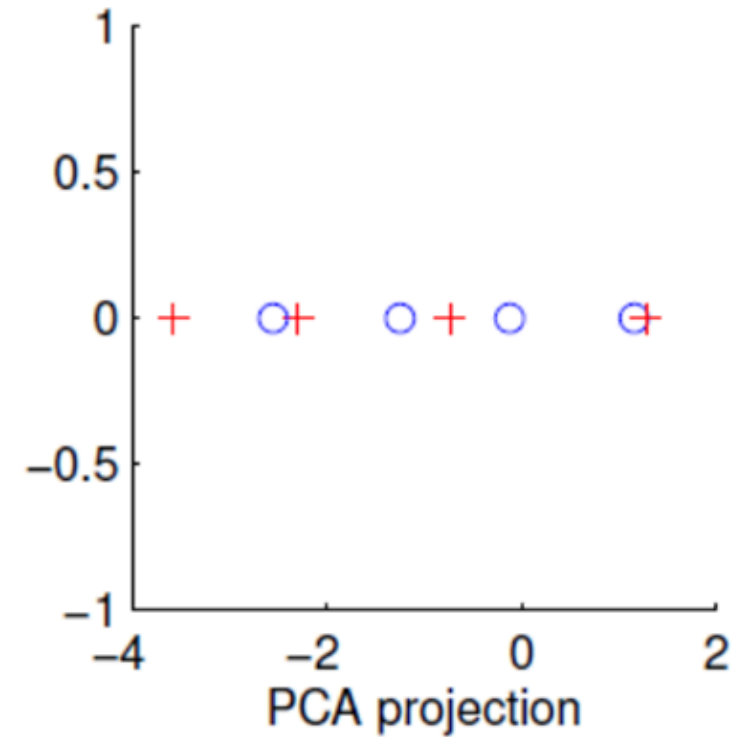
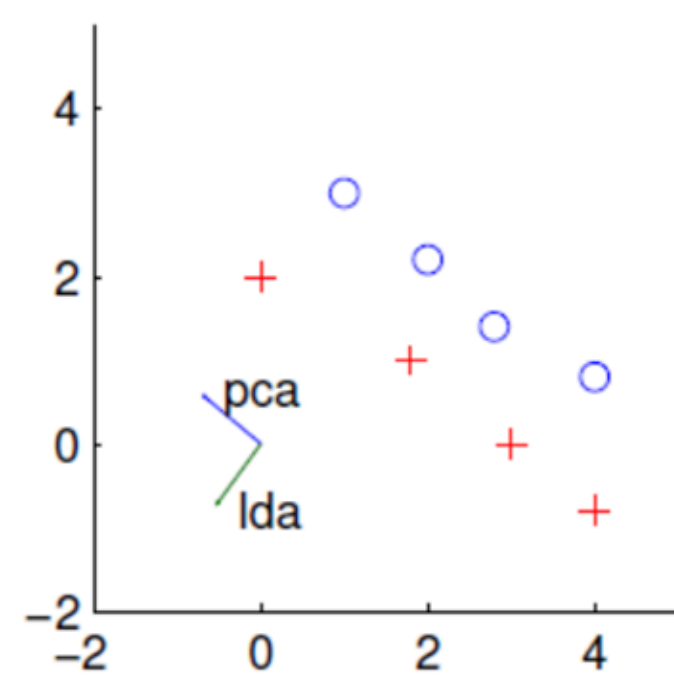
- Find a low-dimensional space such that when x is projected, classes are well-separated
- Find W that maximizes

$$J(w) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2}$$

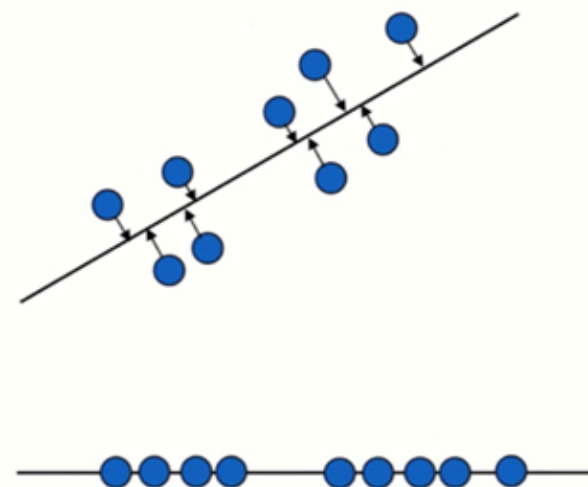
- 정답 label을 잘 분리해내는 축 찾기
- 두 집단의 공분산 행렬이 같다고 가정
- 그룹 수를 미리 정한다.



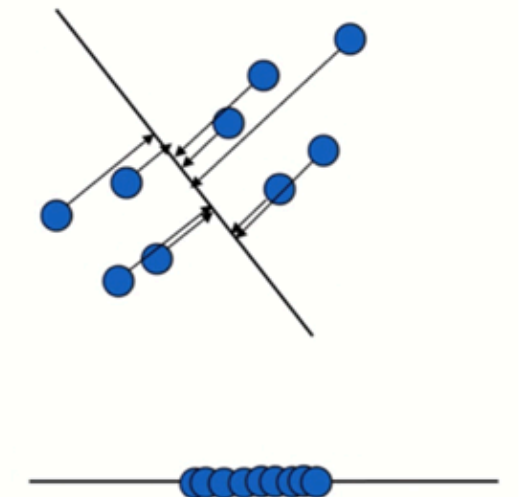
PCA vs LDA



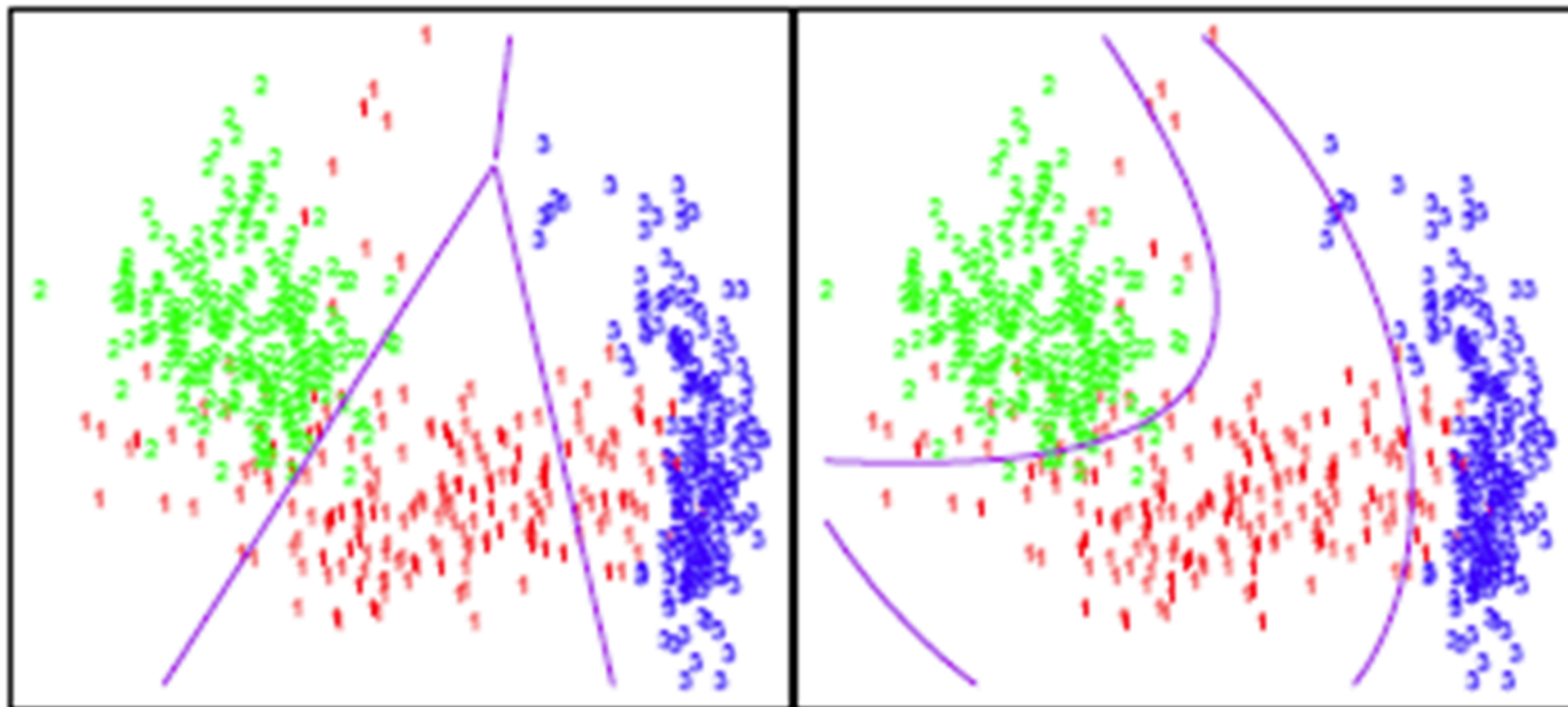
Find the new axis that
maximizes the variance of data



Find the new axis that
minimizes the variance of data



QDA (“Quadratic Discriminant Analysis”)



- 두 집단의 공분산 행렬이 같지 않을 때 이용!
- 직선으로 나누는 LDA와 달리 2차식으로 나누기

Other Algorithms

- tSNE, UMAP → 생명과학 분야에 많이 쓰임
 - Canonical Correlation Analysis (CCA)
 - Isometric feature mapping(Isomap)
 - multi-dimensional scaling (MDS)
 - Locally linear Embedding(LLE)
 - Sufficient Dimension Reduction (SDR)
-
- kPCA(kernel PCA)
 - FPCA (Functional PCA)



SVM

Keywords

Intro

Objectives

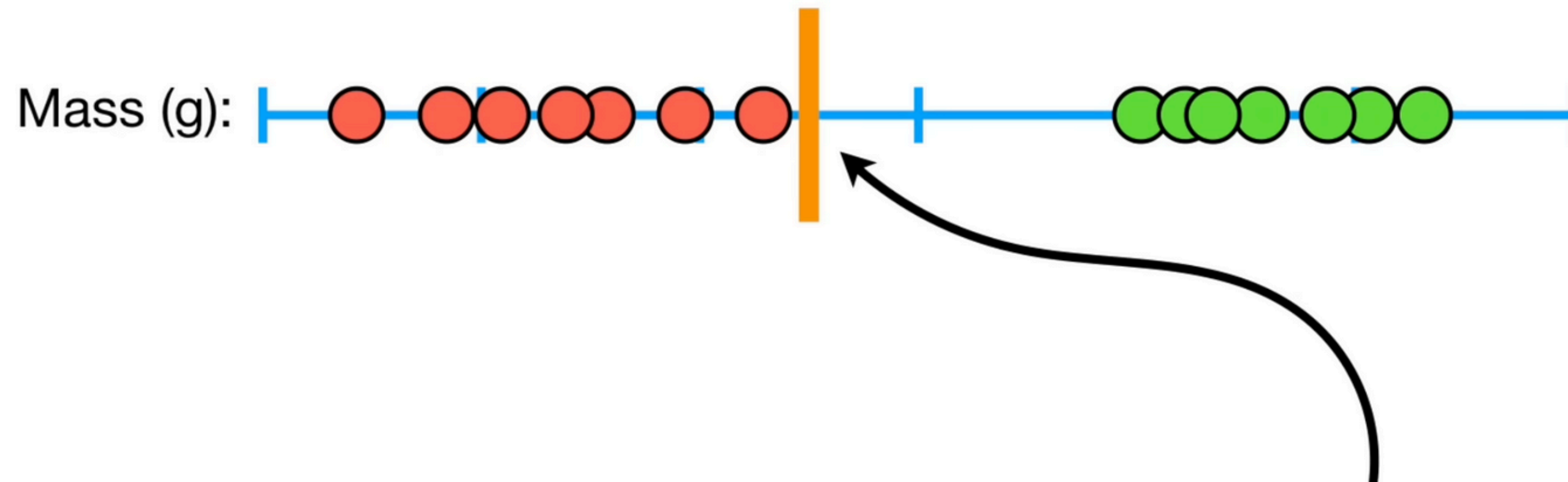
SVM and Kernel Tricks

Support Vector Machine?

- Supervised Learning
- Classification (& Regression)

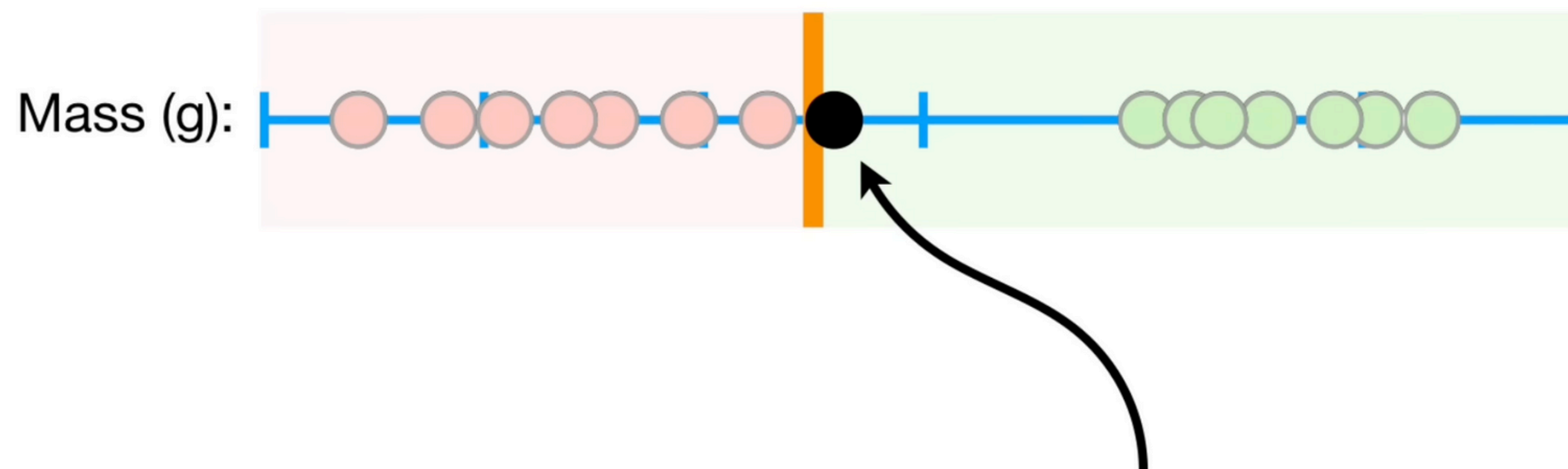
Point	Feature 1	Feature 2	...	Category
1	0.97	0.27		
2	0.77	0.19		
3	0.33	0.85		
4	0.41	0.93		
5	0.27	0.32		

Support Vector Machine?



Based on these observations, we can pick a threshold...

Support Vector Machine?



Support Vector Machine?

Mass (g):



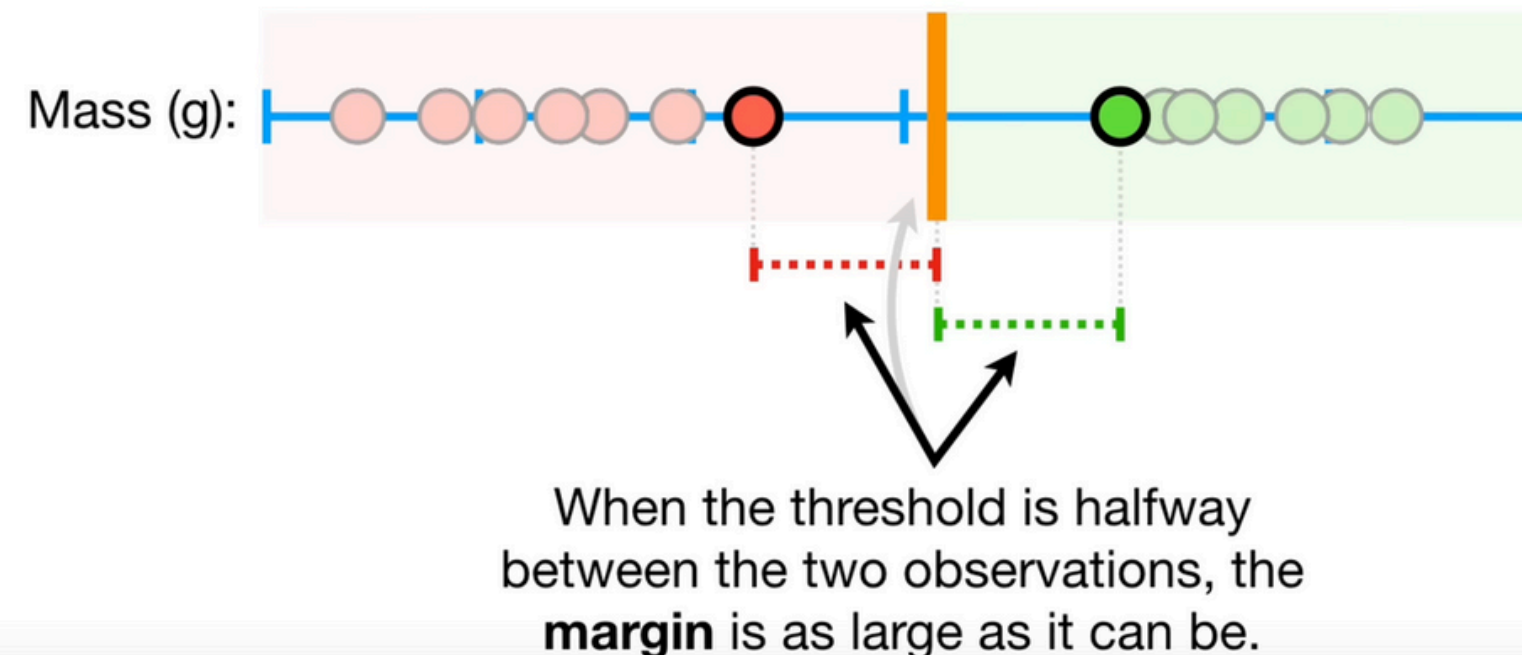
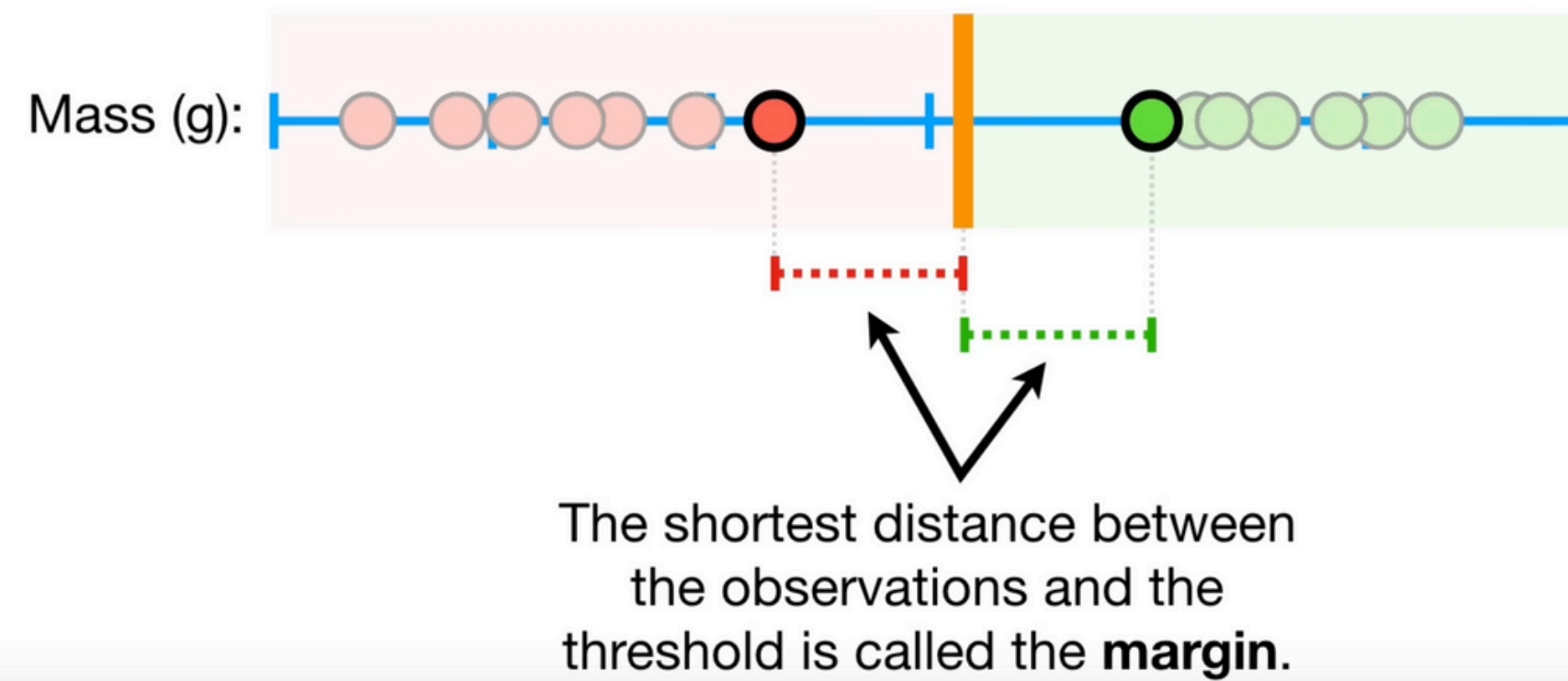
...we can focus on the observations on the edges of each cluster...

Mass (g):



...and use the midpoint between them as the threshold.

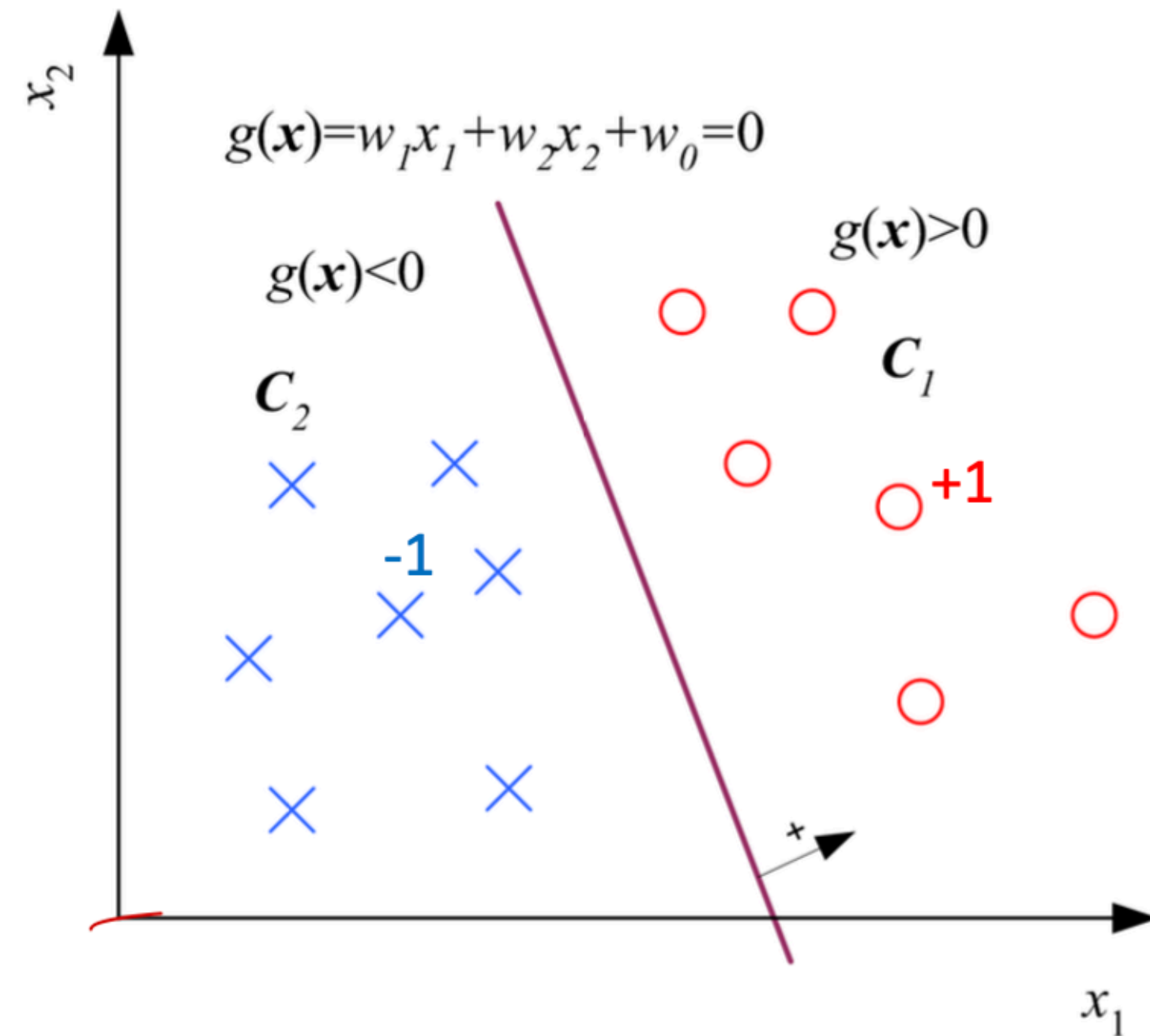
Support Vector Machine?



Support Vector Machine?

두 분류 사이에 존재하는 “여백을 최대화”하는 게 목적!

Linear Discriminant



Decision Boundary : $g(x) = \beta_0 + \beta^T x = 0$

$$X = \{x_i, y_i\} \mid y_i = \begin{cases} +1 & \text{for class } C_1, \\ -1 & \text{for class } C_2 \end{cases}$$

$$\beta^T x + \beta_0 \geq +1, \text{ for } y_i = +1$$

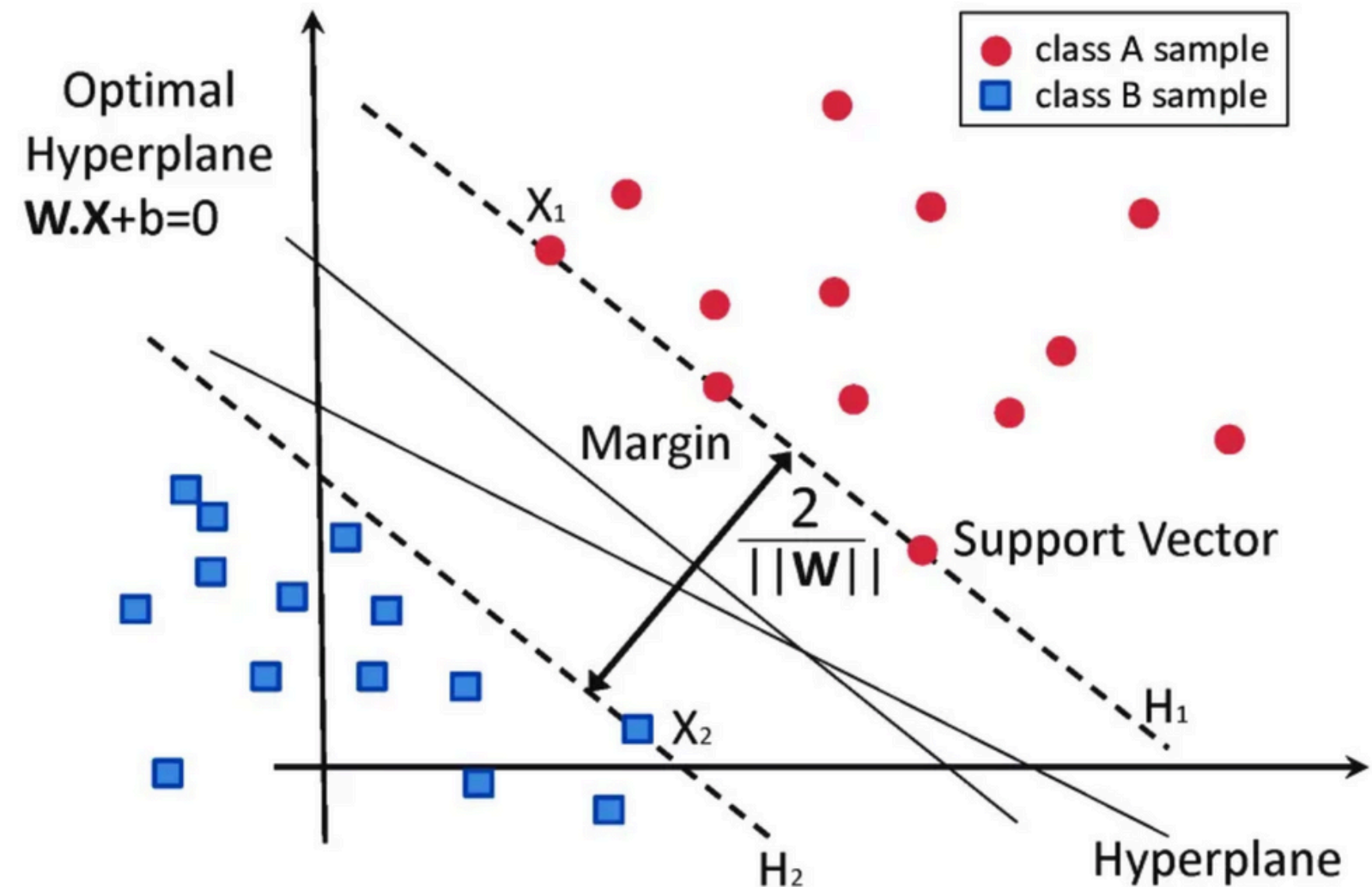
$$\beta^T x + \beta_0 \leq -1, \text{ for } y_i = -1$$

**Classification/Decision Boundary
or Separating Hyperplane**

Support Vector Machine?

- Supervised Learning
- Classification (& Regression)

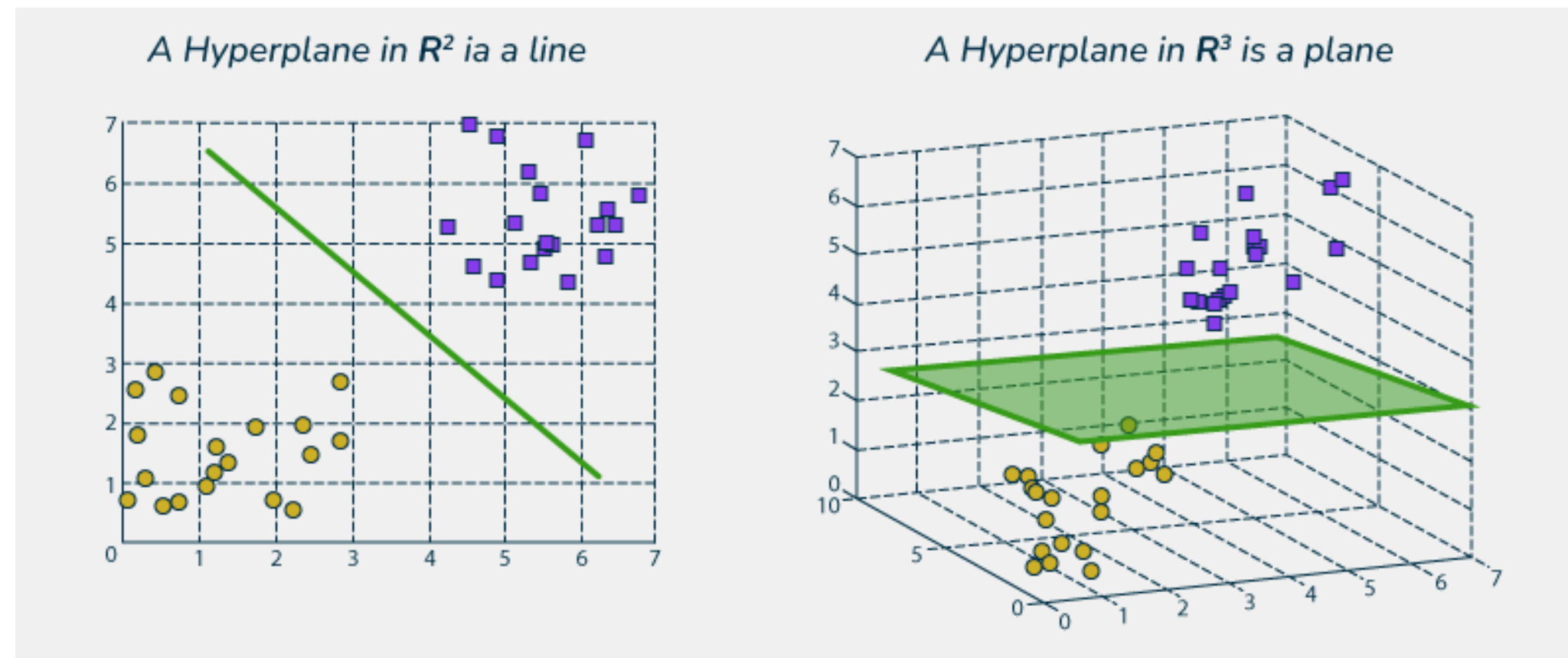
Point	Feature 1	Feature 2	...	Category
1	0.97	0.27		
2	0.77	0.19		
3	0.33	0.85		
4	0.41	0.93		
5	0.27	0.32		



Hyperplane

In a p -dimensional space, a hyperplane is a flat affine subspace of dimension $p-1$
N차원 공간에서 한 차원 낮은 $N-1$ 차원의 부분 공간 ; N차원 공간을 두 개로 나누는 $N-1$ 차원의 부분공간

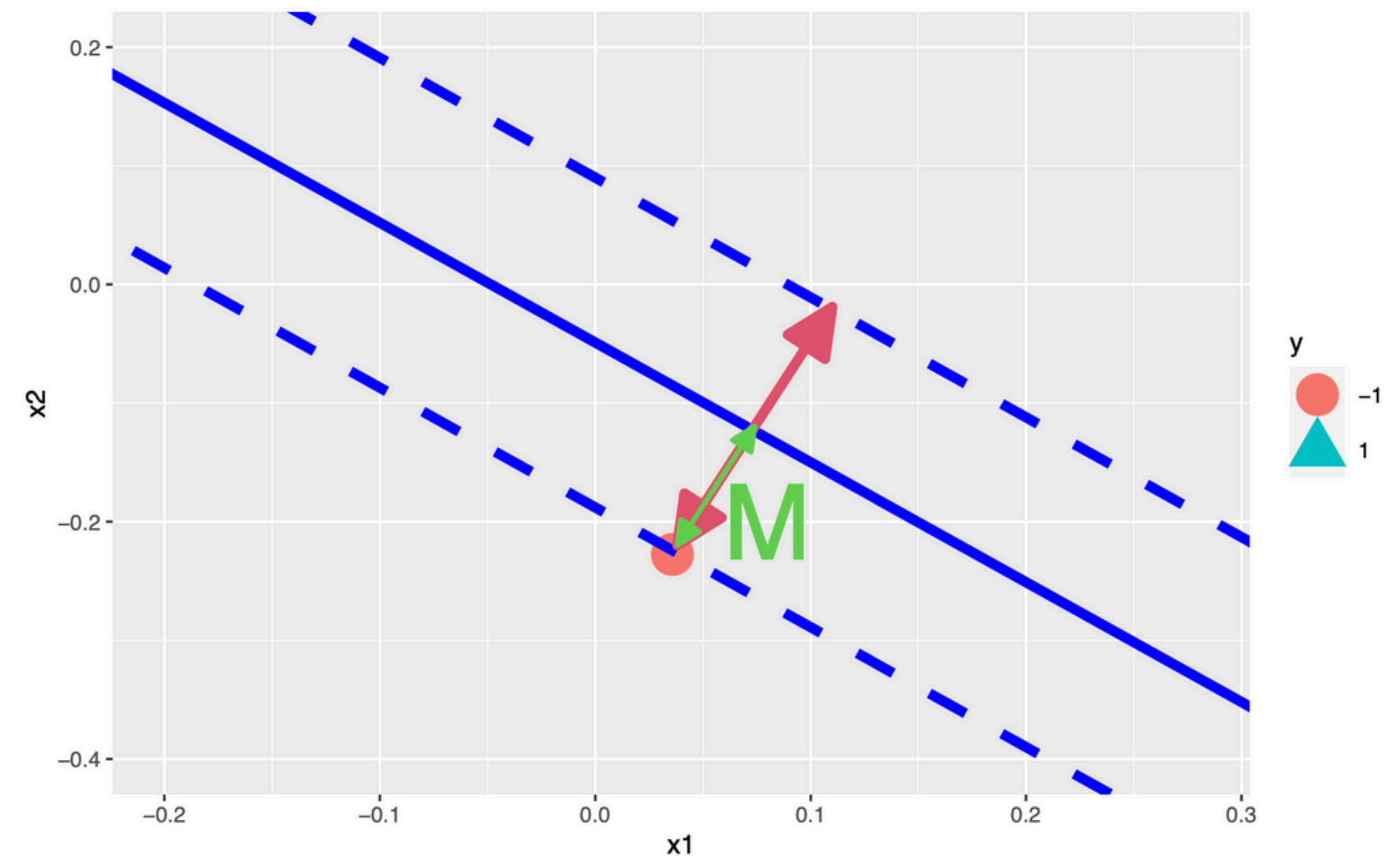
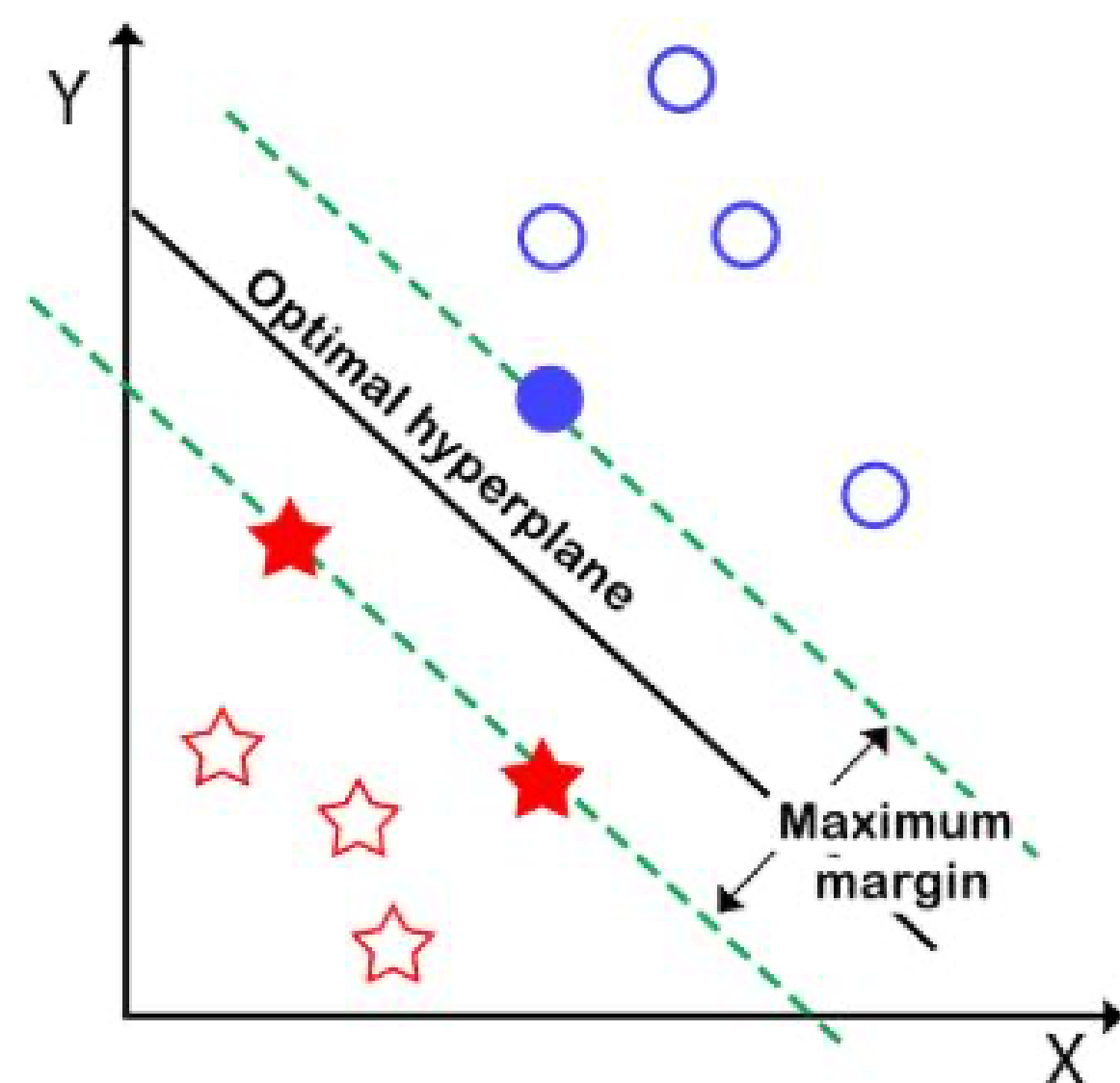
Hyperplanes are decision boundaries that help classify the data points. Data points falling on either side of the hyperplane can be attributed to different classes. Also, the dimension of the hyperplane depends upon the number of features. If the number of input features is 2, then the hyperplane is just a line. If the number of input features is 3, then the hyperplane becomes a two-dimensional plane.



Optimal Hyperplane

Margin : Discriminant로부터 가장 가까운 Instance (data point) 까지의 거리

“Optimal Hyperplane (Discriminant) maximizes Margin !”



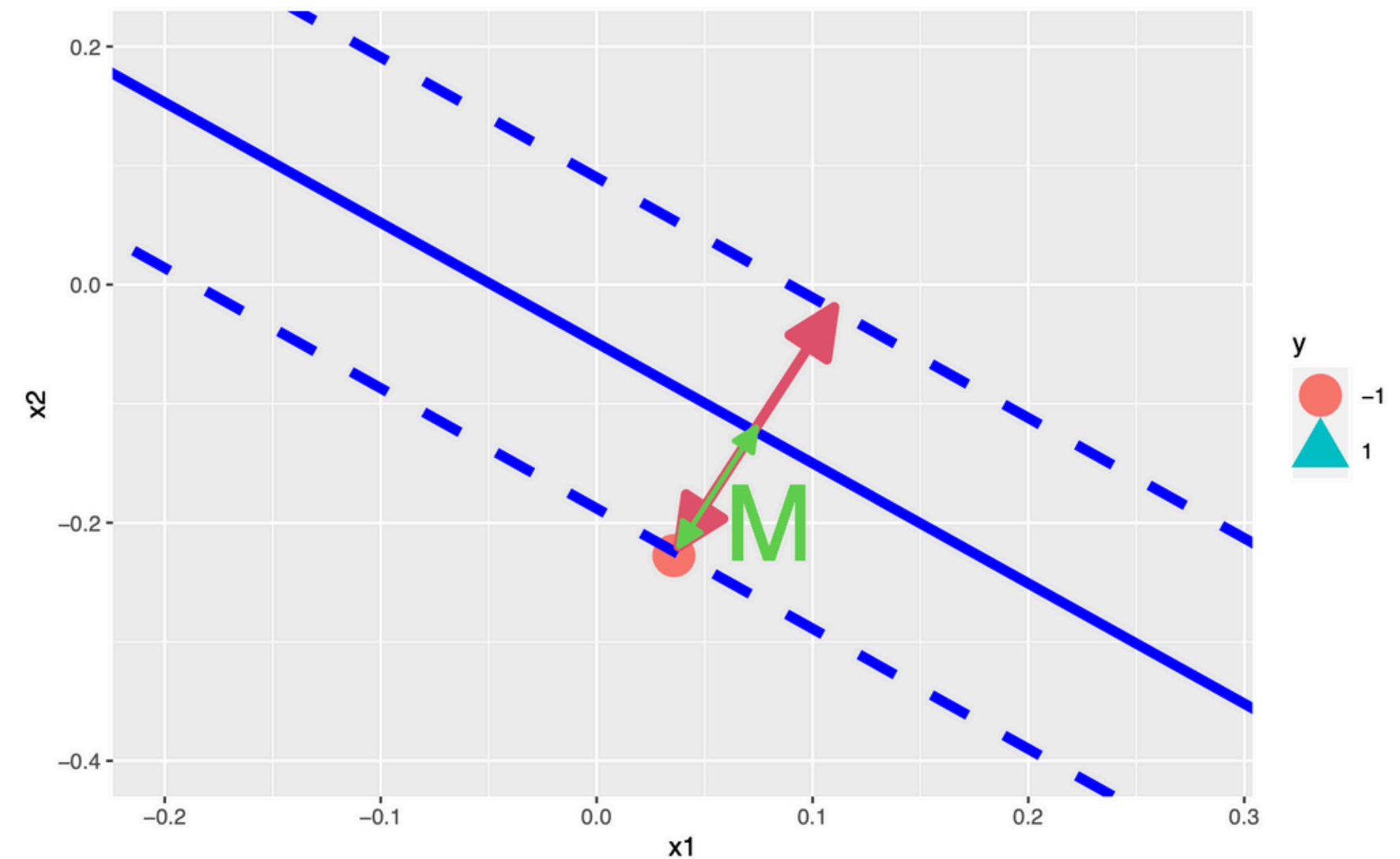
Objective of SVM

Distance x to the hyperplane $g(x)$

Decision Boundary : $g(x) = \beta_0 + \beta^T x = 0$

$$X = \{x_i, y_i\} \mid y_i = \begin{cases} +1 & \text{for class } C_1, \\ -1 & \text{for class } C_2 \end{cases}$$

$$\Rightarrow y_i(\beta_0 + \beta^T x_i) \geq +1, \forall i$$



Maximal Margin Classifier

- maximal margin hyperplane (aka maximal optimal separating hyperplane), which is the separating hyperplane that is farthest from the training observations!

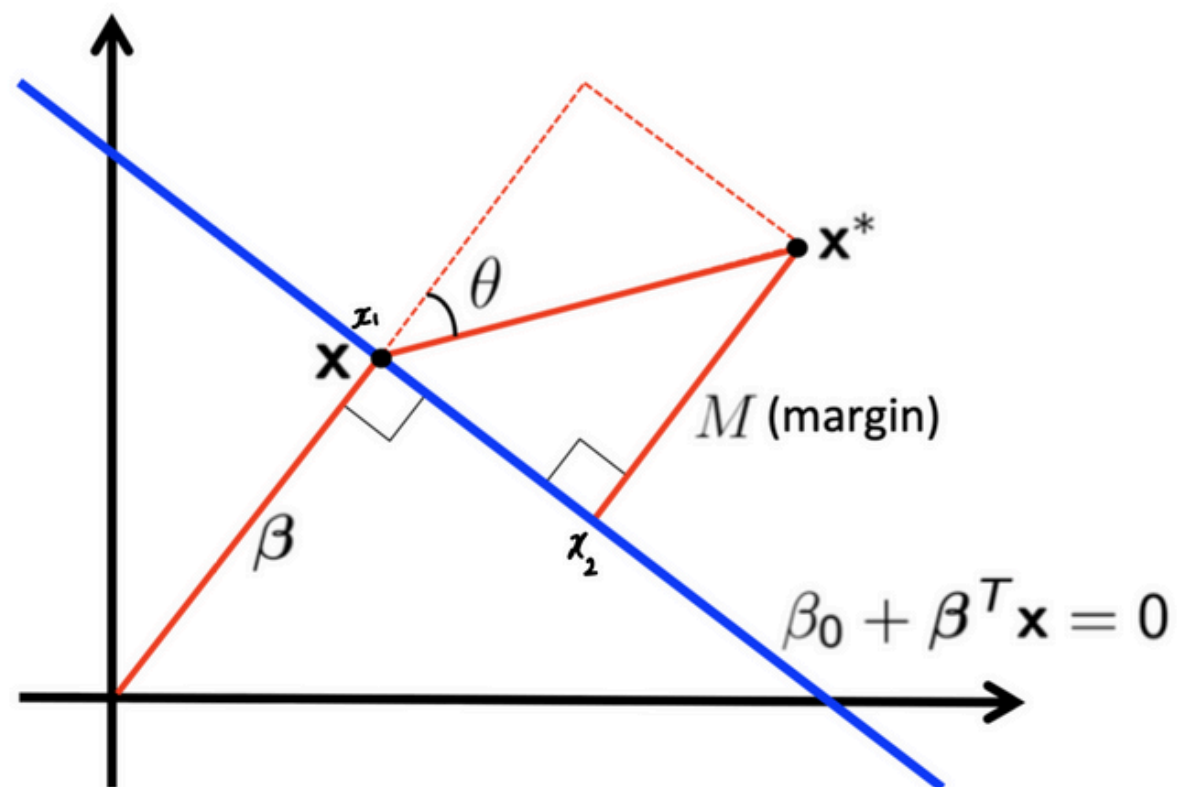


Figure 1: Geometric Margin

- Objective Function :

$$\max_{\beta_0, \beta} M$$

$$\text{subject to } y_i(\beta_0 + \beta^T x_i) \geq M, i = 1, \dots, n;$$

$$\|\beta\| = 1$$

※ The assumption “ $\|\beta\| = 1$ ” is only for β to be unique, and thus can be changed.

**Support vectors: 결정경계와 가장 가까이에 있는(초평면 위에 있는) 데이터 포인트
→ Margin을 결정**

Maximal Margin Classifier

- 목적식 최적화

- Let's assume that $\|\beta\| = 1/M$, so that the margin M is standardized as 1.
- We then have

$$\beta_0 + \beta^T \mathbf{x}^* = \|\beta\| M = 1$$

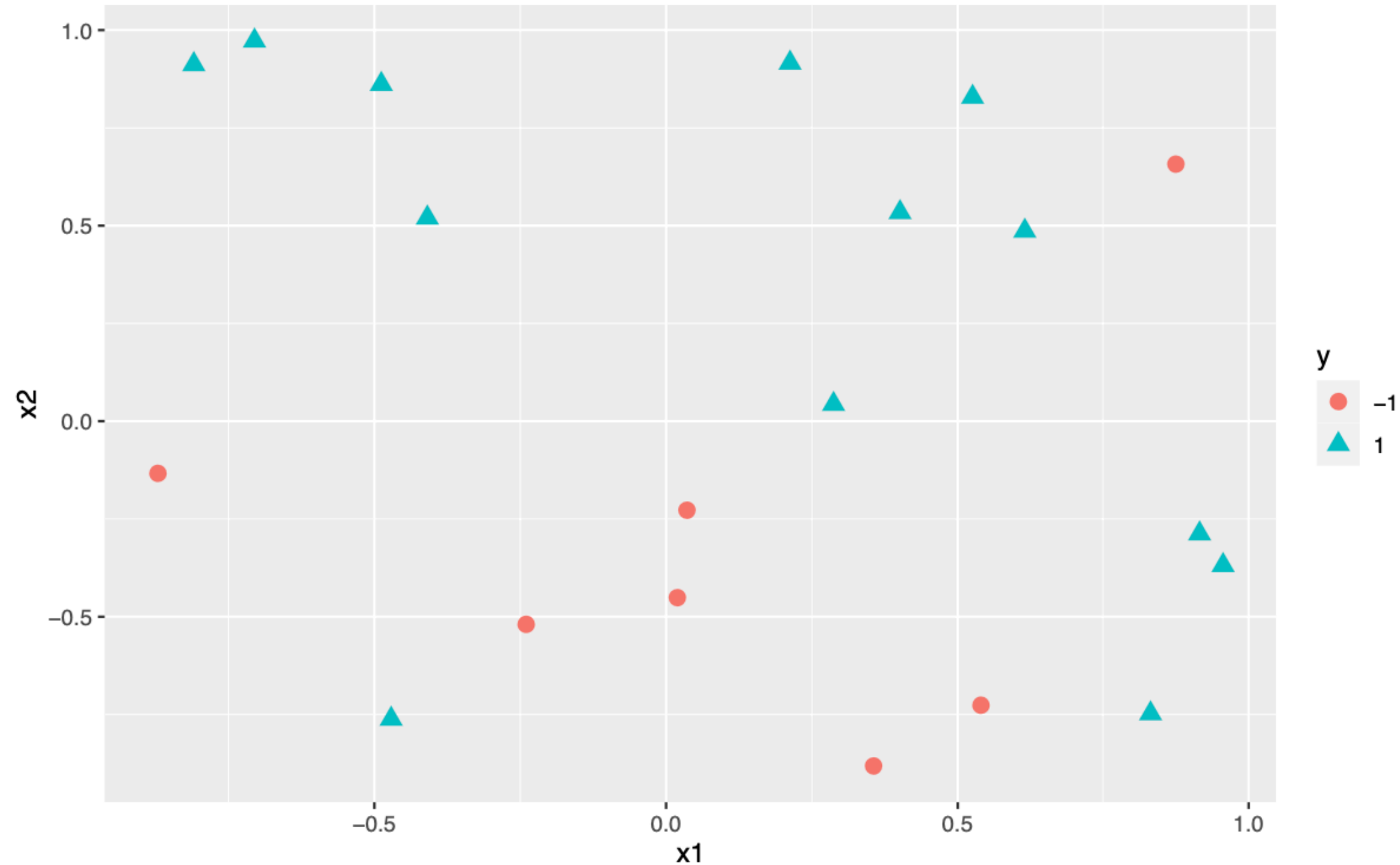
- Maximizing M is equivalent to minimizing $\|\beta\|^2$.
- Equivalent from under $\|\beta\| = 1/M$:

$$\min_{\beta_0, \beta} \frac{1}{2} \beta^T \beta$$

$$\text{subject to } y_i(\beta_0 + \beta^T \mathbf{x}_i) \geq 1, \quad i = 1, \dots, n.$$

- This is known as **Maximal Margin Classifier**.

What if Non-Separable?



In this case, the optimization problem has no solution with $M > 0$!

What if Non-Separable?

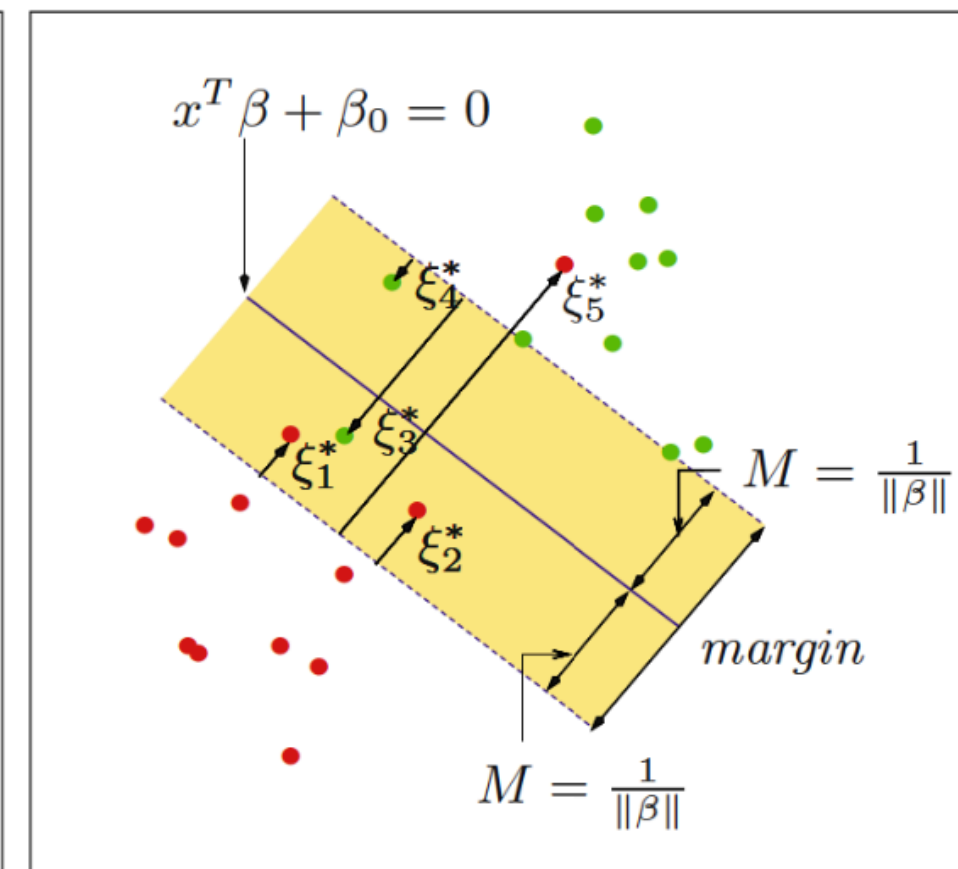
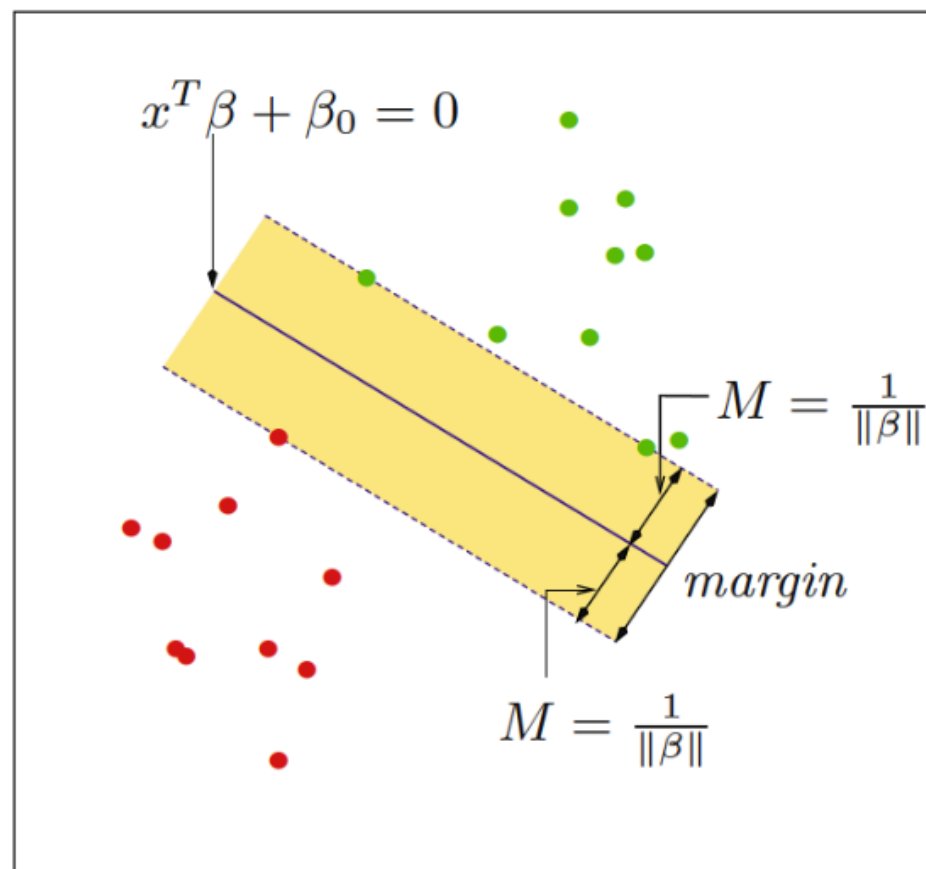
- No solution that satisfies

$$y_i(\beta_0 + \beta^T x_i) \geq 1, \forall i$$

⇒ **Soft Margin! 오류를 허용하자 !**

⇒ **Slack Variable ξ & penalty C 도입**

- Let's relax the constraints by introducing slack variables $\xi_i \geq 0$, and add penalty C for the violations.



Soft Margin Classifier

- 일부 샘플의 오분류를 허용하는 방식
- a.k.a Support Vector Classifier
- New Optimization Problem :

$$\min_{\beta_0, \beta, \xi_i} \beta^T \beta + C \sum_{i=1}^n \xi_i$$

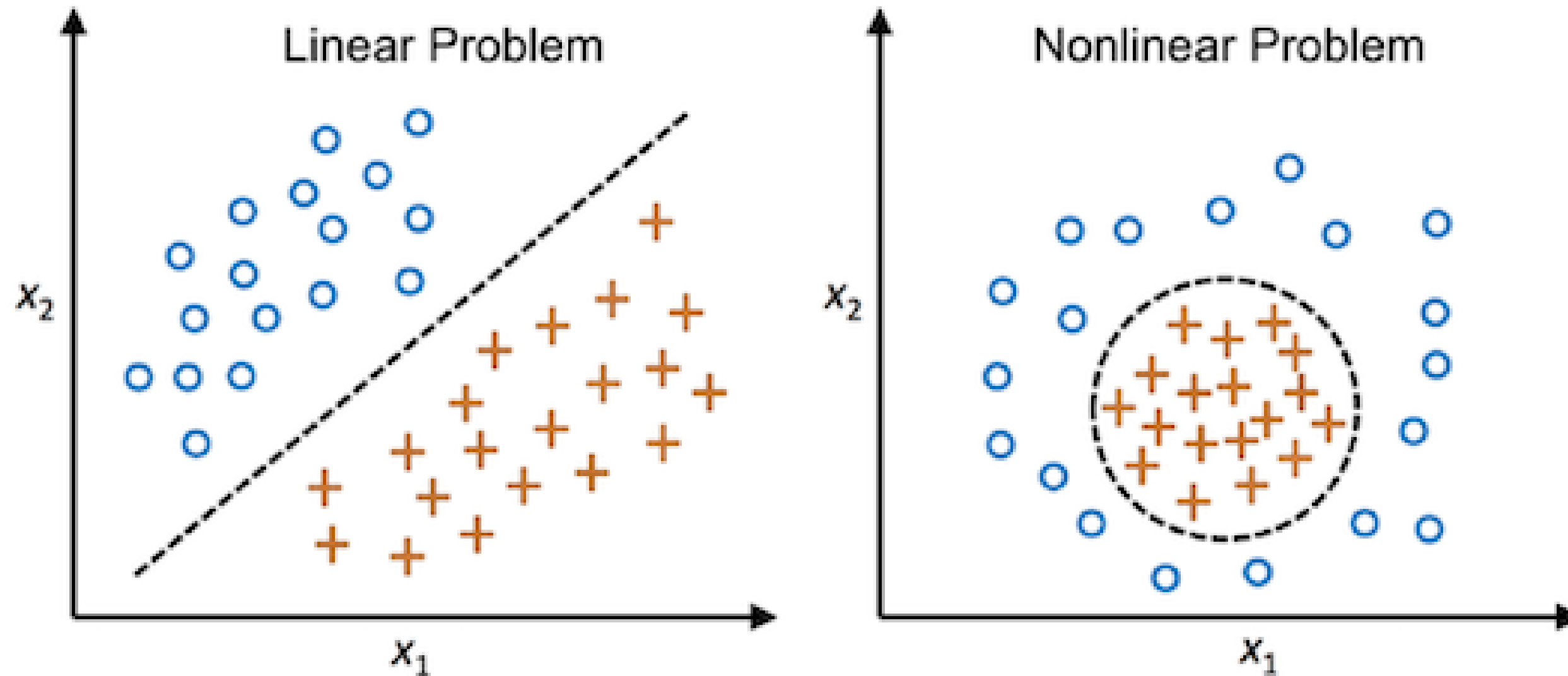
$$\text{subject to } y_i(\beta_0 + \beta^T \mathbf{x}_i) \geq 1 - \xi_i, \quad i = 1, \dots, n$$

$$\xi_i \geq 0, \quad i = 1, \dots, n.$$

- $C \uparrow \rightsquigarrow$ Hard Margin과 유사 (Overfitting 위험)
- $C \downarrow \rightsquigarrow$ Margin의 유연한 조정 (Underfitting 위험)

Extension to Non-Linearity

- 대부분의 데이터는 완벽하게 선형 분리되지 않음



Kernel Trick

$$z = \{1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2\} = [z_1 \ z_2 \ z_3 \ z_4 \ z_5 \ z_6]$$

$$g(z) = w^T z + w_0$$

$$z = \phi(x)$$

$$g(x) = w^T \phi(x) + w_0$$

- Let $\phi(x) = (\phi_1(x), \dots, \phi_n(x))$ from x be a nonlinear map from x to a more general space(i.e., feature space):
- ϕ 를 도입함으로써, 우리는 feature space의 linear decision boundary를 찾을 수 있게 된다 (i.e., space of ϕ)

In Linear SVM...

$$g(x) = w_0 + \sum_t \alpha^t r^t x_t^T x$$

New Feature Space

$$\rightarrow g(z) = w_0 + \sum_t \alpha^t r^t z_t^T z$$

$$g(x) = w_0 + \sum_t \alpha^t r^t \phi(x^t)^T \phi(x)$$

Using Kernel Trick :

$$K(x^t, x)$$

Kernel Trick

- 그런데, Decision Function f on space of $\mathbf{x}$:
$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n y_i \alpha_i \langle \mathbf{x}_i, \mathbf{x} \rangle$$
- Feature Space 상에서 Decision Function은,
$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n y_i \alpha_i \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle$$

Thus, what we need is not the feature ϕ , but its inner product which we call kernel:

$$K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

Given kernel K , we have

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n y_i \alpha_i K(\mathbf{x}_i, \mathbf{x})$$

연산량 증가 없이 비선형 분류 가능 !

Kernel Trick

- 직접 고차원으로 변환하지 않고, 내적(inner product) 만 계산하여 고차원에서의 분류 수행

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n y_i \alpha_i K(\mathbf{x}_i, \mathbf{x})$$

- Kernel Functions

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

Linear Kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma(\mathbf{x}_i - \mathbf{x}_j)^T(\mathbf{x}_i - \mathbf{x}_j))$$

*Gaussian Kernel
(Radial Basis function)*

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma + \gamma \mathbf{x}_i^T \mathbf{x}_j)^p$$

polynomial Kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(k_1 \mathbf{x}_i^T \mathbf{x}_j + k_2)$$

Sigmoid Kernel

SVM with more than Two Classes

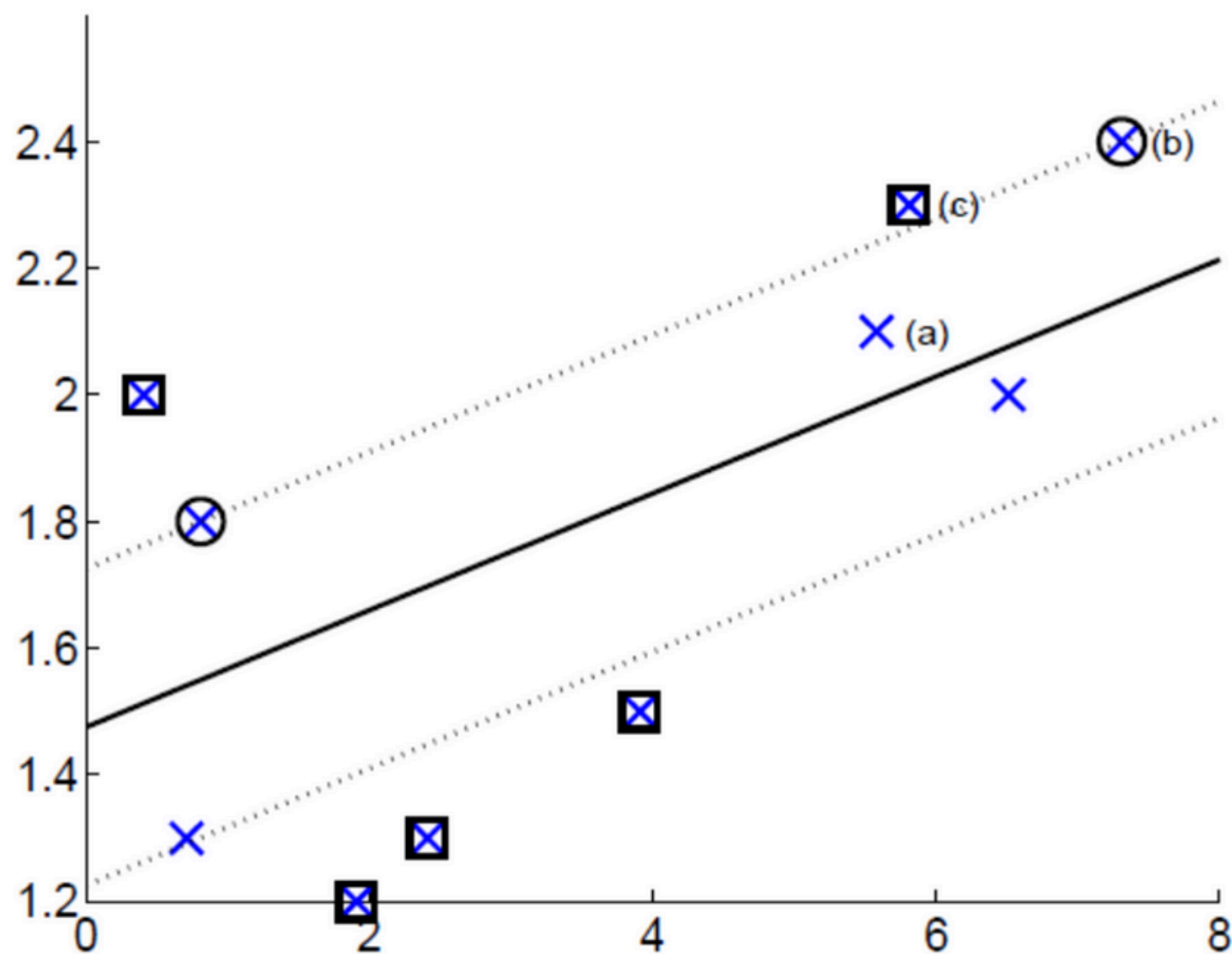
- **One-versus-one Classification**

- 모든 가능한 클래스 쌍 $\binom{K}{2}$ 개에 대해 Binary Classification(SVM)
- 클래스 수가 비교적 적을 때 특히 유리
- Prediction
 - 각 분류기가 두 클래스 중 하나에 Voting
 - 최종 결과는 Majority Voting으로 결정

- **One-versus-all Classification**

- classification각 클래스에 대해 Binary Classification, K개의 SVM fitting
- 선택된 클래스는 양성, 나머지 K-1개 클래스는 음성으로 처리
- Prediction
 - 각 분류기가 해당 클래스일 가능성을 점수로 출력
 - 가장 높은 점수를 받은 클래스를 최종 결과로 채택

SVM – Regression (SVR)



Assume a linear model,

$$f(x) = w^T x + w_0$$

- Error function(loss)

$$e = \begin{cases} 0 & \text{if } |r^t - f(x^t)| < \epsilon \\ |r^t - f(x^t)| - \epsilon & \end{cases}$$

- Lagrangian Method

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_t (\xi_t^+ + \xi_t^-),$$

subject to :

$$r^t - (\mathbf{w}^T \mathbf{x}^t + w_0) \leq \epsilon + \xi_t^+,$$

$$(\mathbf{w}^T \mathbf{x}^t + w_0) - r^t \leq \epsilon + \xi_t^-,$$

$$\xi_t^+, \xi_t^- \geq 0.$$

최대한 Margin 내로 들어오도록 학습
 ... Margin 밖에 있는 Error를 최소화!

SVM – Regression

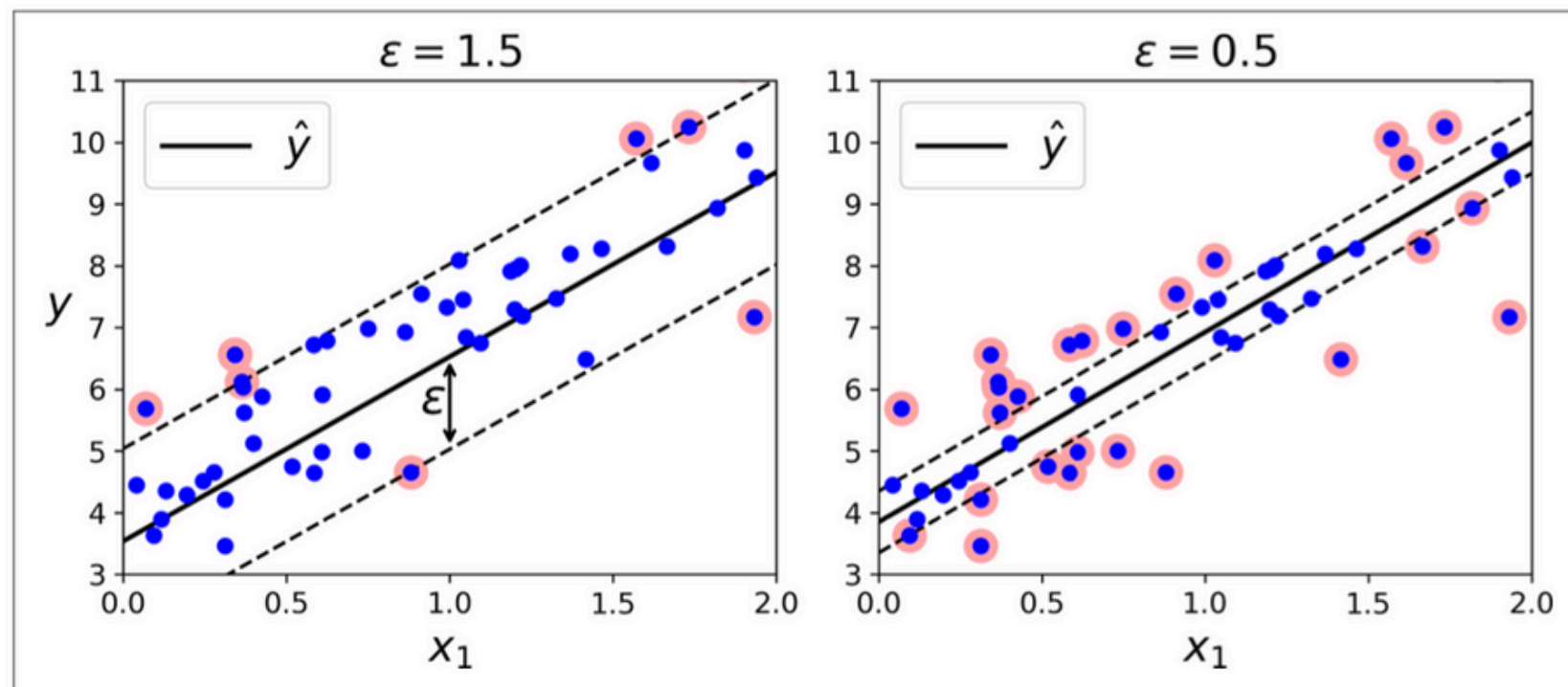
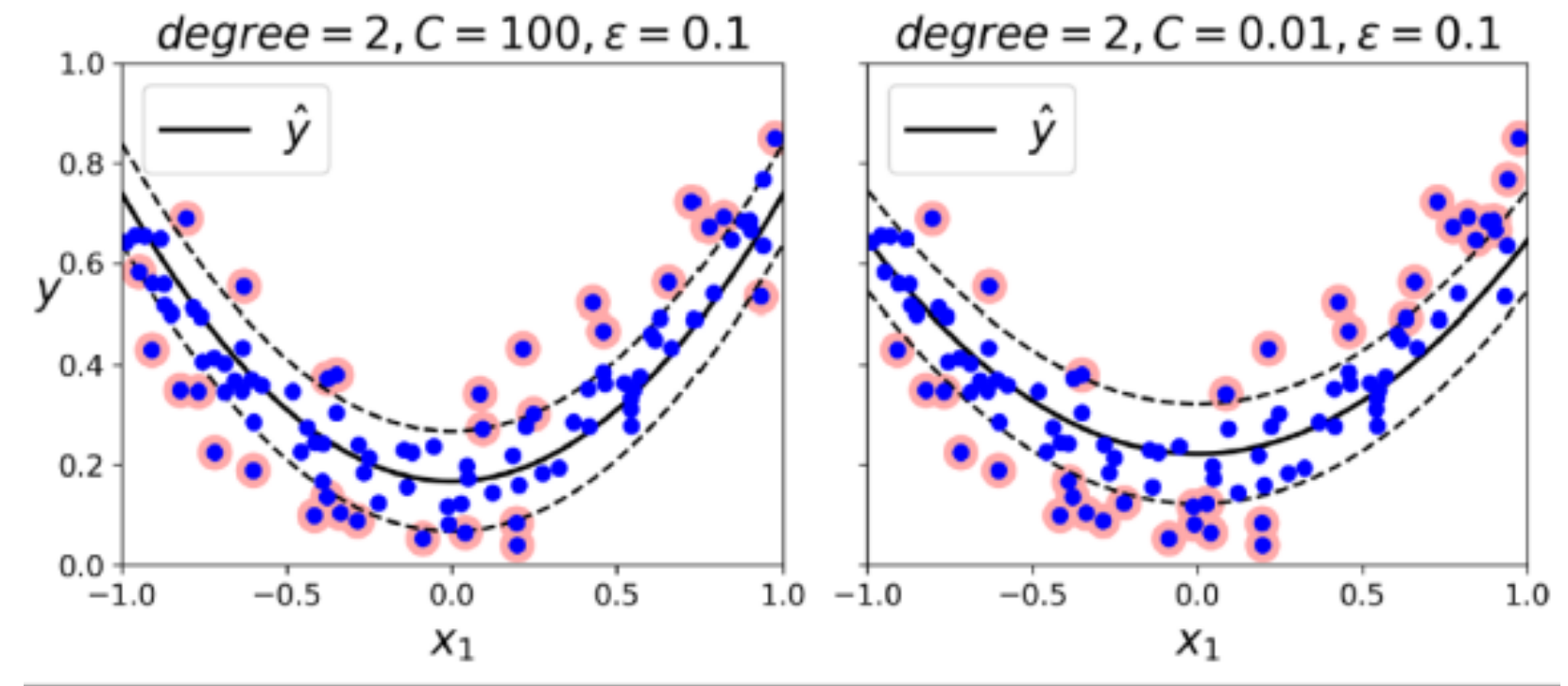


Figure 5-10. SVM Regression

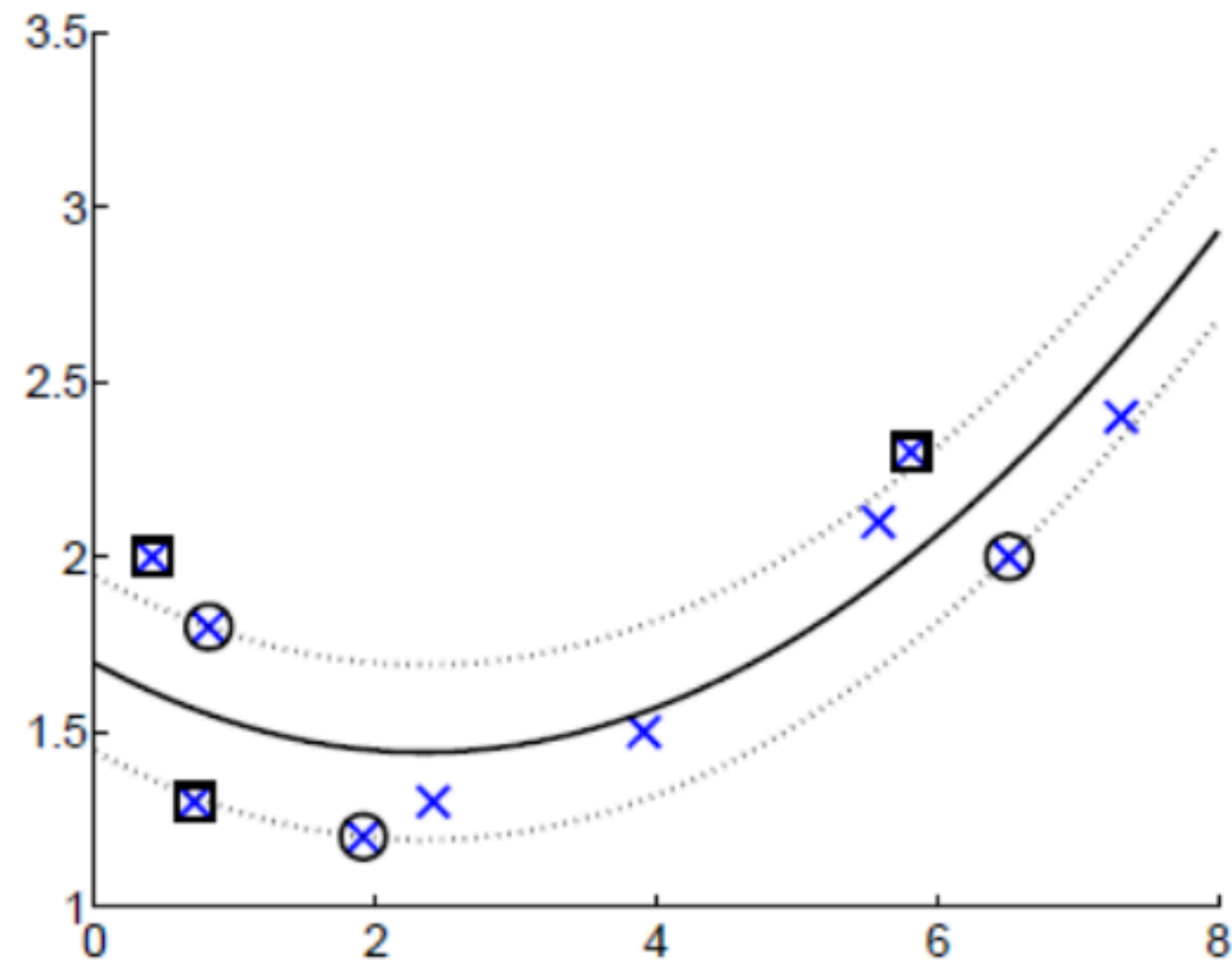


- ϵ : Margin의 폭을 결정
- 값이 클수록 마진 내부 데이터에 대해 관대.
- ϵ 이 작으면 더 많은 데이터가 마진 외부로 간주, 슬랙 변수(오차) $\uparrow$

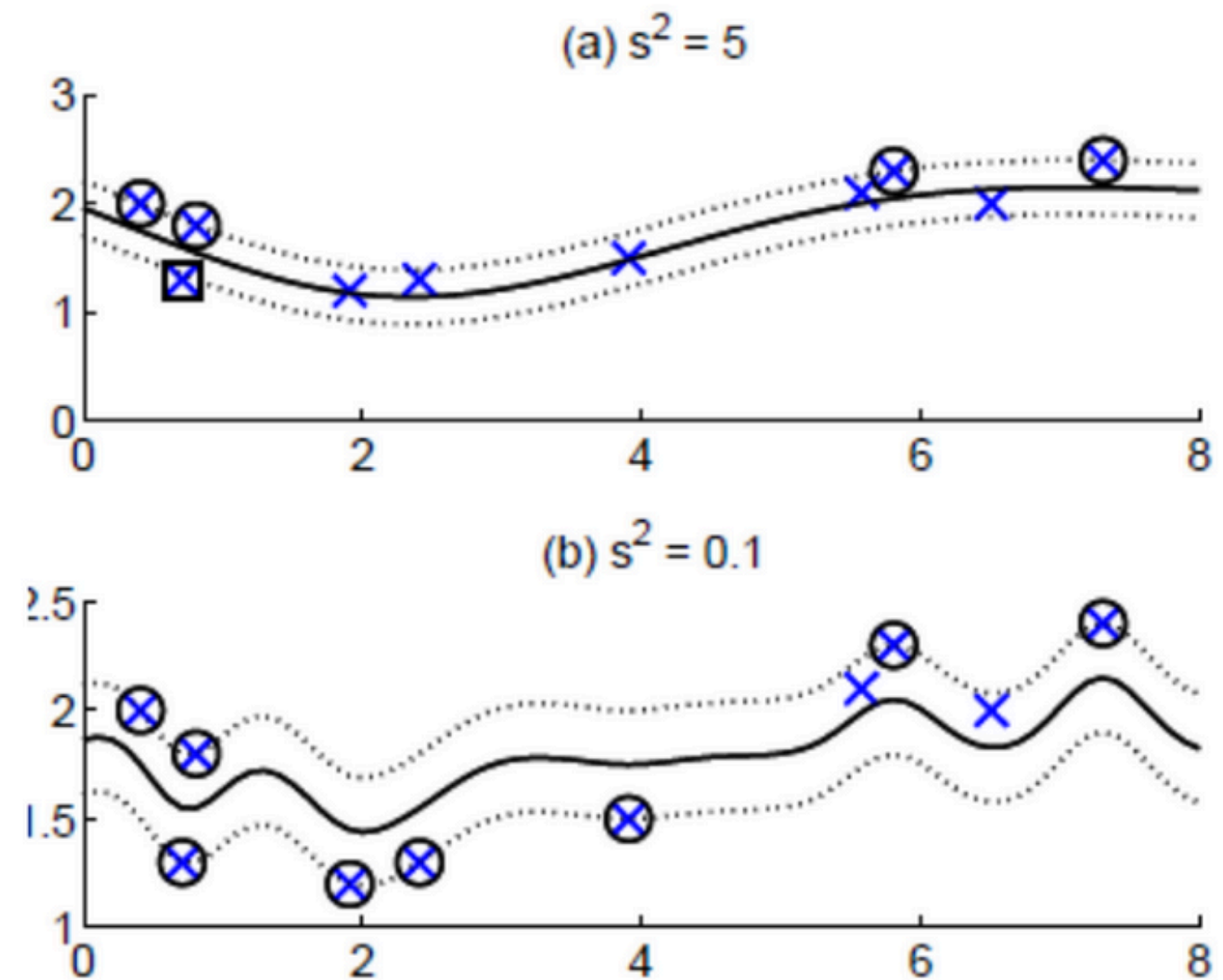
- C : 페널티 파라미터, 오차의 허용 정도
- 높은 C 값 $\rightarrow$ Margin 외부의 오차에 큰 페널티, 과적합 위험
- 낮은 C 값 $\rightarrow$ 오차에 관대, Underfitting 위험

SVM – Kernel Regression

- Polynomial Kernel



- Gaussian Kernel





해당 자료는 Slack & KUBIG github에서 보실 수 있습니다 :)

5주차 과제는 2/11 수요일까지 github에 업로드 해 주시기 바랍니다!

5주차 과제 : KUBIG_2026_SPRING/Basic Study/ML/5주차
파일명 2026_1_ML_week5_{본인 이름}.ipynb 로 제출